

PrimeTime Workshop Lab Guide

中文学习译本

版本：2018.06

Synopsys Customer Education Services

原文版权归 Synopsys, Inc. 所有（PrimeTime Workshop Lab Guide, 10-I-034-SLG-015, 2018.06）。

本文件为非官方个人学习译本，仅供学习参考，不得用于商业传播。

命令名、选项名、文件名、消息编号等技术标识符保持英文原文不译。

目录

版权声明与专有信息	iv
目的地管制声明	iv
免责声明	iv
商标	iv
第三方链接	iv
 第一章 Lab 1: PrimeTime 流程	 1
1.1 概述	1
1.1.1 相关文件与目录	2
1.2 操作说明	2
1.2.1 任务 1: 恢复 PrimeTime 会话	2
1.2.2 任务 2: [可选] 探索有用命令	3
1.2.3 任务 3: 验证已有的 PrimeTime 会话	5
1.2.4 任务 4: 执行运行脚本并分析运行结果	6
1.2.5 任务 5: 分析 STA 报告	7
 第二章 Lab 2: 约束方法论	 10
2.1 概述	10
2.1.1 相关文件与目录	10
2.2 操作说明	11
2.2.1 任务 1: 验证约束	11
2.2.2 任务 2: 分析输入延迟约束的时序报告	12
2.2.3 任务 3: 分析输出延迟约束的时序报告	14
 第三章 Lab 3: 生成报表	 17
3.1 概述	17
3.1.1 相关文件与目录	17
3.2 操作说明	18
3.2.1 任务 1: 为 Lab 3 配置 PrimeTime	18
3.2.2 任务 2: 生成汇总报告	18
3.2.3 任务 3: 分析 Setup 与 Hold 时序报告	20
3.2.4 任务 4: 使用正确的时序报告开关	22
3.2.5 任务 5: 识别半时钟周期路径	23

第四章	Lab 4: 约束多时钟	25
4.1	概述	25
4.1.1	相关文件与目录	25
4.2	操作说明	26
4.2.1	任务 1: 熟悉设计时钟	26
4.2.2	任务 2: 使用 GUI 报告时钟关系	27
4.2.3	任务 3: 使用 GUI 探索时序路径细节	29
4.2.4	任务 4: 报告虚假违例	30
4.2.5	任务 5: 重新执行运行脚本以减少违例	33
第五章	Lab 5: 额外约束	34
5.1	概述	34
5.1.1	相关文件与目录	34
5.2	操作说明	35
5.2.1	任务 1: 调试 PTE-070 信息消息	35
5.2.2	任务 2: 探索 Time Borrow 与 Latch	36
第七章	Lab 7: 基于路径的分析 (PBA)	40
7.1	概述	40
7.1.1	相关文件与目录	40
7.2	操作说明	41
7.2.1	任务 1: 报告当前设计中的违例路径	41
7.2.2	任务 2: 执行 path mode PBA	41
7.2.3	任务 3: 执行 exhaustive PBA	42
7.2.4	任务 4: [可选] 生成 GBA 与 PBA 对比汇总报告	43
第八章	Lab 8: SI 延迟分析	44
8.1	概述	44
8.1.1	相关文件与目录	44
8.2	操作说明	45
8.2.1	任务 1: 执行 PrimeTime SI 运行脚本	45
8.2.2	任务 2: 在 Shell 中生成并解读 SI 报告	46
8.2.3	任务 3: 在 GUI 中生成 SI 报告	49
第九章	Lab 9: SI 噪声分析	50
9.1	概述	50
9.1.1	相关文件与目录	50
9.2	操作说明	51
9.2.1	任务 1: 运行 PrimeTime SI 噪声分析	51
9.2.2	任务 2: 在 Shell 中解读噪声报告	52
9.2.3	任务 3: 在 GUI 中运行噪声分析	54

附录 A Job Aid 速查卡	56
A.1 PrimeTime JOB AID: Timing Reports	56
A.1.1 实用 PrimeTime 命令	56
A.1.2 违例汇总报告	57
A.1.3 report_timing 开关	58
A.1.4 Tcl 示例	58
A.2 PT JOB AID: Run Scripts	59
A.2.1 实用 Tcl 过程 (SolvNet)	59
A.2.2 .synopsys_pt.setup 示例	59
A.2.3 variables.tcl 示例	59
A.2.4 查找非默认应用变量	60
A.2.5 Path-Based Analysis	60
A.2.6 RUN.tcl 示例	60
A.2.7 从 Unix Shell 运行	61
A.3 其他 Job Aid 页 (PDF p.117–124)	61
A.3.1 Clocks and More (约 PDF p.117)	61
A.3.2 Constraints / Interface (约 PDF p.118–119)	62
A.3.3 PrimeTime SI: Crosstalk Delay Reports (约 PDF p.119–120)	62
A.3.4 PrimeTime SI Crosstalk Delay: Fine Tuning (PDF p.121)	63
A.3.5 PrimeTime SI: Noise Analysis (约 PDF p.122–123)	64
A.3.6 空白页 (PDF p.124)	65

版权声明与专有信息

Copyright Notice and Proprietary Information

版权声明与专有信息

© 2018 Synopsys, Inc. 保留所有权利。本软件与文档包含属于 Synopsys, Inc. 的保密专有信息。软件与文档根据许可协议提供，仅可依照许可协议条款使用或复制。未经 Synopsys, Inc. 事先书面许可，或未在许可协议中明确允许时，不得以任何形式、通过任何电子、机械、手工、光学或其他方式复制、传输或翻译本软件与文档的任何部分。

目的地管制声明

Destination Control Statement

本出版物中的全部技术数据受美利坚合众国出口管制法律约束。违反美国法律向其他国家国民披露信息是被禁止的。读者有责任确定适用法规并予以遵守。

免责声明

Disclaimer

SYNOPSYS, INC. 及其许可方对本材料不作任何明示或默示担保，包括但不限于适销性与特定用途适用性的默示担保。

商标

Trademarks

Synopsys 及某些 Synopsys 产品名称为 Synopsys 的商标, 详见 <http://www.synopsys.com/Company/Pages/Trademarks.aspx>。

所有其他产品名或公司名可能是其各自所有者的商标。

第三方链接

Third-Party Links

本文档中包含的第三方网站链接仅为方便读者。Synopsys 不认可这些网站，也不对其做法（包括隐私做法）、可用性与内容负责。

Synopsys, Inc.

690 E. Middlefield Road

Mountain View, CA 94043

www.synopsys.com

文档订购号 (Document Order Number): 10-I-034-SLG-015

PrimeTime Lab Guide

Chapter 1

Lab 1: PrimeTime 流程

Lab 1: PrimeTime Flow

学习目标

完成本实验后，你应该能够：

- 恢复（restore）先前保存的 PrimeTime 会话（session）；
- （可选）使用若干有助于交互式效率的 PrimeTime 命令，并学会如何查找命令、变量与设计库相关信息；
- 验证已恢复的 save-session；
- 练习推荐的 PrimeTime 流程（flow）；
- 解读建立时间（setup）与保持时间（hold）时序报告中的关键组成部分。

实验时长

约 45 分钟。

1.1 概述

Overview

本实验大致步骤如下：

1. 恢复一个 PrimeTime 会话；
2. 学习若干有用命令（可选任务）；
3. 验证已恢复的保存会话，并练习推荐流程；
4. 解读建立时间与保持时间时序报告。

1.1.1 相关文件与目录

Relevant Files and Directories

本实验全部文件位于主目录下的 `lab1_flow` 目录中。

路径 / 文件	说明
<code>lab1_flow/</code>	当前工作目录
<code>common_setup.tcl</code>	多工具共享的 setup 文件
<code>pt_setup.tcl</code>	PrimeTime 专用 setup 文件
<code>pt_scripts/</code>	运行脚本目录
<code>pt.tcl</code>	运行脚本 (run file)
<code>.synopsys_pt.setup</code>	自动读取的 PT setup 文件
<code>orca_savesession</code>	已保存会话目录
<code>RUN.tcl</code>	用于重建 <code>orca_savesession</code> 的运行脚本

1.2 操作说明

Instructions

目标：通过验证已恢复的 save session、练习推荐流程，并分析 setup/hold 报告，熟悉 PrimeTime。

1.2.1 任务 1：恢复 PrimeTime 会话

Task 1. Restore a PrimeTime Session

调用先前保存的 PrimeTime 会话以执行静态时序分析 (STA, Static Timing Analysis)。

1. 从实验 Unix 目录 `lab1_flow` 启动 PrimeTime:

```
unix% cd lab1_flow
unix% pt_shell
```

2. 恢复先前保存的 PrimeTime 会话。该步骤将读入设计网表 (netlist)、库 (libraries) 与约束 (constraints)。此后设计即可进行分析。

注意

- 下文的 `orca_savesession` 是一个 Unix 目录。
- 如需重建该会话，可使用：
`pt_shell -f RUN.tcl | tee -i run.log`
- 执行 `RUN.tcl` 时出现的任何 **PARA-124** 错误，在本课程实验中可安全忽略。
- PrimeTime 支持命令、选项、变量与文件名补全：键入少量字符后按 **Tab** 键即可。

```
pt_shell> restore_session orca_savesession
```

3. 生成覆盖率分析报告：

```
pt_shell> report_analysis_coverage
```

问题

问题 1. 当前分析的设计名称是什么？

答：设计名称为 **ORCA**。

问题 2. ORCA 有多少个 setup 违例与 hold 违例？

答：共有 **23** 个 setup 违例与 **53** 个 hold 违例。

4. 生成全局时序报告：

```
pt_shell> report_global_timing
```

问题

问题 3. 其中有多少个 reg-reg 类型的 setup 与 hold 违例？

答：类型为 reg-reg 的违例为：**9** 个 setup、**53** 个 hold。

1.2.2 任务 2: [可选] 探索有用命令

Task 2. [OPTIONAL TASK] Explore Helpful Commands

1. 执行以下三条历史快捷命令：

```
pt_shell> history
pt_shell> !!
pt_shell> !2
```

问题

问题 4. 说明上面最后两条历史命令的区别。

答：命令 !2 重复历史中第 2 条已执行命令；命令 !! 重复历史中最近一条已执行命令。

2. 也可用上下箭头在历史事件列表中滚动，作为上一步的替代方式。

执行以下命令查看默认 emacs 编辑模式下的全部键绑定：

```
pt_shell> list_key_bindings
```

3. 体验 page mode 别名：执行下列命令生成会滚出屏幕的长报告：

```
pt_shell> report_timing -group [get_path_group *]
```

4. 打开 page mode:

```
pt_shell> page_on
pt_shell> !rep
```

5. 用空格键翻页阅读长报告；在 page mode 下键入 q 可退出长报告。若要关闭 page mode，使用别名 page_off。

6. 使用 Tcl 过程 view，把时序报告送到单独窗口：

```
pt_shell> view report_timing -group [get_path_group *]
```

7. 查找用于恢复 PrimeTime 会话的命令，并显示该命令的帮助信息：

```
pt_shell> help restore*  
pt_shell> man restore_session  
pt_shell> restore_session -help
```

注意

以下是显示语法帮助的另一方式：

```
pt_shell> help -v restore_session
```

问题

问题 5. 根据上一条命令，`restore_session` 是否接受开关（switches）？

答：是。它接受会话名等参数/开关，从而可指定恢复众多已保存会话中的哪一个。

8. PrimeTime 中的时间单位由主工艺库（main technology library）决定。要查找 ORCA 的时间单位，先列出内存中的全部库。

注意

报告中库名前的 * 表示主库（main library）。

```
pt_shell> list_lib
```

9. 对主库生成报告以查看时间单位。

注意

建议复制粘贴库名以免拼写错误。时间单位位于报告最顶部。

```
pt_shell> report_lib cb13fs120_tsmc_max
```

问题

问题 6. ORCA 设计的时序报告（以及所有其他报告）使用的时间单位是什么？

答：1 ns。

注意

别忘了在 page mode 下用 q 退出长报告，无需读完整份报告即可回到 `pt_shell` 提示符。

10. 显示当前设计使用的单位：

```
pt_shell> report_units
```

1.2.3 任务 3：验证已有的 PrimeTime 会话

Task 3. Validate an Existing PrimeTime Session

本任务验证已读入 PrimeTime 的输入：当前设计与库、反标（back-annotation）与约束。

1. 确认当前设计为顶层模块 **ORCA**：

```
pt_shell> current_design
```

2. 将 Unix 路径上的库与已读入 PrimeTime 的库进行对照：

```
pt_shell> printvar search_path
pt_shell> printvar link_path
pt_shell> list_libraries
```

问题

问题 7. link_path 中的 4 个库是否都已成功读入 PrimeTime？

答：是——用 link_library 指定的四个库与 list_libraries 显示的一致。

问题 8. 哪一个库定义了时间单位、工作条件（operating conditions）以及其他延迟计算相关信息的默认值？

答：cb13fs120_tsmc_max——list_libraries 输出中库名前的星号表示“主库”。

问题 9. 使用的时间单位是什么？

答：

```
pt_shell> report_lib cb13fs120_tsmc_max
...
Time Unit : 1ns

pt_shell> report_units
...
Time_unit : 1e-09 Second
```

3. 验证网线是否已完整反标：

```
pt_shell> report_annotated_parasitics
```

问题

问题 10. 是否存在未反标的网？

答：是：存在一些内部无驱动网（driverless nets），以及一些边界 pin-to-pin 网未反标。

问题 11. 作为调试缺失网的“下一步”，report_annotated_parasitics 的哪个选项比较合适？

答：report_annotated_parasitics -help 可列出选项；-list_not_annotated 可列出网名；若关注 pin-to-pin 网，可使用 -pin_to_pin_nets。

4. 验证设计是否已完整约束：

```
pt_shell> check_timing
```

问题

问题 12. 作为调试缺失约束的“下一步”，`check_timing` 的哪个选项比较合适？

答：`check_timing -help` 列出选项；`-verbose` 可列出未约束端点（unconstrained endpoints）。

5. 验证单元中的检查是否被充分行使（exercised）；并思考可能原因：

```
pt_shell> report_analysis_coverage
pt_shell> report_case_analysis
```

问题

问题 13. 许多时序检查处于 `untested` 状态——这是否合理？

答：合理——`test_mode`、`scan_en` 与 `power_save` 被约束为关；对额外模式做 STA 可能使更多检查变为 `tested`。

6. 退出 PrimeTime：

```
pt_shell> quit
```

1.2.4 任务 4：执行运行脚本并分析运行结果

Task 4. Execute the Run Script and Analyze the Run

1. 执行运行脚本，并将结果记录到日志文件 `run.log`：

```
UNIX> pt_shell -f ./pt_scripts/pt.tcl | tee -i run.log
```

问题

问题 14. 执行运行脚本过程中是否出现错误？

答：没有。该运行脚本的设计使得一旦出错会在中途终止；若脚本完整跑完，则执行过程中未发生错误。此外，日志末尾由 `quit` 给出的 `Diagnostics Summary` 也显示本次运行无错误。

2. 若有错误，须先处理错误，再进入下一步。
3. 评估日志文件：用文本编辑器打开日志，搜索 `update_timing` 相关消息（`UITE-214`）、`print_message_info` 输出，以及 `quit` 输出。然后在 `profile` 目录中查看文件 `tcl_profile_sorted_by_cpu_time`。

问题

问题 15. 哪一步占用的 CPU 时间最多？

答：对本实验而言，`source` 约束占用的 CPU 时间最多。

问题 16. 哪些命令引发了 `UITE-214` 消息？

答: `update_timing -full`, 以及调用宏 `PLL_SHIFT` (其内部触发了 `update_timing`)。

问题 17. 是否有部分 timing update 可以避免?

答: 可以。在 `set_propagated_clocks` 前后各有一次 `update_timing`——之前那一次是不必要的。一次 `update_timing` 位于 `pt.tcl`, 另一次位于约束文件 `orca_pt_other.tcl` (由 `orca_pt_constraints` source, 再由 `pt.tcl` source)。

问题 18. 为什么在通读日志之前, 从 `quit` 命令的输出入手可能是个好起点?

答: 它给出潜在问题点的高层摘要: 警告/信息消息、timing update, 以及性能统计。

1.2.5 任务 5: 分析 STA 报告

Task 5. Analyze STA Reports

为时钟 `SYS_CLK` 生成并解读 `setup` 与 `hold` 两份 STA 报告。

1. 启动 PrimeTime, 并恢复上一任务中保存的会话:

```
unix% pt_shell
pt_shell> restore_session my_savesession
```

2. 显示 ORCA 中的时钟:

```
pt_shell> report_clock
```

问题

问题 19. ORCA 中有多少个时钟?

答: ORCA 中有 6 个时钟。

3. 为时钟 `SYS_CLK` 生成一份“简短 (short)”的 `setup` 时序报告。使用命令行补全 (Tab 键) 展开命令以及选项 `-group` 与 `-path`:

```
pt_shell> report_timing -group SYS_CLK -path short
```

注意

- 数据到达 (data arrival) 段中包含数据通路单元及其延迟的行被去掉, 因此报告称为 “short”。
- 上述命令默认生成 `setup` 报告。

问题

问题 20. 至少有 4 处线索表明这是 `setup` 而非 `hold` 报告。你能找出多少?

答: 最醒目的线索是报告中的 **library setup time**。更细微的线索包括:

- 数据到达与数据要求所用的时钟边分别为 0 ns 与 8 ns (而非 `hold` 时的 0 ns 与 0 ns);
- 报告头中的 Path Type 为 **max**, 表示 `setup` (**min** 表示 `hold`);
- 数据到达时间晚于数据要求时间, 且 `slack` 违例 (若为 `hold`, 在类似情形下 `slack` 往往表

现为不同关系)。

问题 21. 写出起点与终点触发器的实例名。

答:

```
Startpoint: I_ORCA_TOP/I_BLENDER/s3_op2_reg[18]
(rising edge-triggered flip-flop clocked by SYS_CLK)
Endpoint:   I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]
(rising edge-triggered flip-flop clocked by SYS_CLK)
Path Group: SYS_CLK
Path Type:  max
```

问题 22. 该时序路径的时钟偏斜 (clock skew) 为 0.511 ns; 可用报告中的哪两行计算得到?

答: 可用启动端与捕获端的 clock network delay (propagated) 两行: $3.247 - 2.736 = 0.511$ ns。

问题

问题 23. 该时钟偏斜如何影响松弛 (slack) ——有助于还是有害于 slack?

答: 在本条 setup 报告中, 时钟偏斜有害于 slack: 启动端触发器的时钟延迟使数据晚到约 0.511 ns, 从而加大 setup 违例。

问题 24. 违例相对于时钟周期有多大?

答: 时钟周期为 8 ns, 违例约 0.987 ns, 大约相当于周期的 12%。

4. 生成 hold 时序报告。

下列快捷方式会执行历史中以字母 **rep** 开头的最近一条命令, 并追加开关 **-delay min** (从而生成 hold 报告):

```
pt_shell> !rep -delay min
```

问题

问题 25. 至少有 4 处线索表明这是 hold 而非 setup 报告。你能找出多少?

答: 最醒目的线索是报告中的 **library hold time**。更细微的线索包括:

- 数据到达与数据要求所用时钟边均为 0 ns;
- Path Type 为 **min**, 表示 hold;
- 数据到达早于数据要求时间, 且 slack 违例。

问题 26. 在本 hold 报告中, 时钟偏斜如何影响 slack——有助于还是有害于 slack?

答: 从报告可见 (施加 clock network delay 之后), 捕获时钟边晚于启动时钟边到达, 这有害于 hold slack, 并导致违例。

5. 退出 PrimeTime:

```
pt_shell> quit
```

至此 Lab 1 完成。请返回课堂讲义。

Chapter 2

Lab 2: 约束方法论

Lab 2: Constraining Methodology

学习目标

完成本实验后，你应该能够：

- 通过检查约束完整性与未测试（`untested`）时序检查来验证约束；
- 在时序报告中识别并解读约束，包括：
 - 时钟约束；
 - 接口约束。

实验时长

约 30 分钟。

2.1 概述

Overview

本实验大致步骤如下：

1. 验证约束；
2. 在时序报告中解读约束。

2.1.1 相关文件与目录

Relevant Files and Directories

本实验全部文件位于主目录下的 `lab2_constraints` 目录中。

路径 / 文件	说明
lab2_constraints/	当前工作目录
orca_savesession/	用于恢复实验的会话目录
.synopsys_pt.setup	自动读取的 PT setup 文件
RUN.tcl	用于重建 orca_savesession 的运行脚本

2.2 操作说明

Instructions

2.2.1 任务 1：验证约束

Task 1. Validate Constraints

1. 从实验 Unix 目录 lab2_constraints 启动 PrimeTime, 并使用 orca_savesession 目录恢复 PrimeTime 会话。

注意

- 如需重建该会话，可使用：
`pt_shell -f RUN.tcl | tee -i run.log`
- 执行 RUN.tcl 时出现的任何 **PARA-124** 错误，在本课程实验中可安全忽略。

```
unix% cd lab2_constraints
unix% pt_shell
pt_shell> restore_session orca_savesession
```

2. 检查约束完整性：

```
pt_shell> check_timing -verbose
```

问题

问题 1. 设计中所有寄存器是否都已有时钟？

答：是。在 **Information: Checking 'no_clock'**. 消息之后，没有报告任何时钟 pin。

问题 2. 是否存在缺失的约束？你能解释原因吗？

答：是。有 2 个 output 端口报告缺失 output delay：

```
Warning: There are 2 endpoints which are not constrained for maximum
delay.
```

```
sd_CKn
sd_CK
```

上述警告可忽略——这 2 个是时钟输出端口（可用 `report_clock` 确认），不应施加 output delay 约束。

3. 检查设计中未测试 (untested) 的时序检查:

```
pt_shell> report_analysis_coverage
```

问题

问题 3. 近三分之二的 setup/hold 检查处于 untested 状态! 造成这一现象的两个原因是什么?

答: **constant_disabled** 与 **false_path**。(可使用 `report_analysis_coverage -status untested -check {setup hold}` 确认。)

问题 4. 为何存在未行使 (unexercised) 的 min_pulse_width 检查?

答: min_pulse_width 检查仅当 pin 上有时钟时才会被行使。由于这些是诸如 set/clear 之类的非时钟异步 pin, 未在其上定义时钟。(可使用 `report_analysis_coverage -status untested -check min_pulse_width` 确认。)

问题 5. setup 与 hold 各有多少个 output delay 约束? 这些约束是满足 (met) 还是违例 (violated)?

答: 从 `report_analysis_coverage` 可见: setup 与 hold 各有 **75** 个 output delay 约束; 其中 setup 有 **9** 个违例, hold 有 **39** 个违例。请确认所有 output 端口均已同时约束 setup 与 hold。

```
pt_shell> restore_session orca_savesession
pt_shell> report_analysis_coverage
```

Type of Check	Total	Met	Violated	Untested
setup	9629	3575 (37%)	13 (0%)	6041 (63%)
hold	9629	3517 (37%)	71 (1%)	6041 (63%)
recovery	1316	1210 (92%)	0 (0%)	106 (8%)
removal	1316	1204 (91%)	6 (0%)	106 (8%)
min_period	20	20 (100%)	0 (0%)	0 (0%)
min_pulse_width	7273	5957 (82%)	0 (0%)	1316 (18%)
out_setup	75	66 (88%)	9 (12%)	0 (0%)
out_hold	75	36 (48%)	39 (52%)	0 (0%)
All Checks	29333	15585 (53%)	138 (0%)	13610 (46%)

2.2.2 任务 2: 分析输入延迟约束的时序报告

Task 2. Analyze a Timing Report For Input Delay Constraint

1. 生成施加于端口 pad[0] 的输入延迟约束报告:

```
pt_shell> report_port -input_delay pad[0]
```

问题

问题 6. 到达 pad[0] 的最小与最大到达时间 (arrival time) 分别是多少?

答: 最小与最大到达时间分别为 **2 ns** 与 **8 ns** (rise 与 fall 数据跳变在该端口使用相同约束值)。

问题 7. 约束 pad[0] 的外部起点时钟名称是什么?

答: 时钟名称为 **PCI_CLK**。

2. 生成从端口 pad[0] 开始的 setup 时序报告, 并据此回答下列问题。可参考标注为 “timing reports” 的 job aid, 回忆 report_timing 的合适开关。

```
pt_shell> report_timing -from pad[0]
```

问题

问题 8. 你使用了时序报告中的哪些行来确认该路径起始于端口 `pad[0]`，且为 `setup` 报告？

答：

```
pt_shell> report_timing -from pad[0]
Startpoint: pad[0] (input port clocked by PCI_CLK)
Endpoint:   I_ORCA_TOP/I_PCI_CORE/d_out_i_bus_reg[0]
            (rising edge-triggered flip-flop clocked by PCI_CLK)
Path Group: PCI_CLK
Path Type:  max
```

问题 9. 列出该时序报告中涉及的全部用户指定约束。

答：时钟周期是一项约束；时钟 `PCI_CLK` 为 `propagated`（非 `ideal`）；输入外部延迟（来自 `input delay` 约束）也是用户指定约束。

问题 10. 对于起点时钟 `PCI_CLK`，时钟延迟（`clock latency`）必须包含在何处？

答：时钟网络延迟为零，因此表示外部时钟延迟的唯一位置是 `input delay` 约束中的输入外部延迟（`input external delay`）。正确建模方式是：对 `set_input_delay` 使用 `-network_latency_included` 与 `-source_latency_included`。

问题 11. 描述端口 `pad[0]` 的方向（即它是 `input`、`output` 还是 `inout` 端口）。

答：端口 `pad[0]` 是 `inout` 端口，因此它既是时序路径起点，也是时序路径终点。

Point	Incr	Path
clock <code>PCI_CLK</code> (rise edge)	0.000	0.000
clock network delay (propagated)	0.000	0.000
input external delay	8.000	8.000 r
<code>pad[0]</code> (inout)	0.000	8.000 r

问题 12. 描述该时序路径的终点（即它是 `output` 端口还是内部触发器）。

答：终点是由 `PCI_CLK` 时钟的上升沿触发 `flip-flop`（实际上是一个具有 `setup/hold` 时序检查的时序模型，外观类似 `flip-flop`）。

- 从同一端口 `pad[0]` 生成另一份 `setup` 报告，同时显示计算得到的时钟网络延迟（`clock network delay`）详情。可参考“`timing reports`”`job aid`，并善用历史命令。

```
pt_shell> !rep -path full_clock
```

问题

问题 13. 对于终点时钟 `PCI_CLK`，时钟源延迟（`clock source latency`）与时钟网络延迟各有多大？

答：下方仅展示时序报告的 `data required time` 段。源延迟（`source latency`）为 **0 ns**；时钟网络延迟为 **0.768 ns**（`15.768 - 15.000`）。

```
pt_shell> !rep -path full_clock
clock PCI_CLK (rise edge)          15.000    15.000
clock source latency                0.000    15.000
pclk (in)                          0.000    15.000 r
...
I_ORCA_TOP/I_PCI_CORE/d_out_i_bus_reg[0]/CP (sdcrq1)
                                0.088    15.768 r
library setup time                 -0.153    15.615
data required time                  15.615
```

问题 14. 时钟 PCI_CLK 在何处定义（时钟定义点）？

答：时钟 PCI_CLK 在输入端口 pclk 处定义。时钟定义点将时钟源延迟与时钟网络延迟分开。

4. 生成从端口 pad[0] 开始的 hold 时序报告：

```
pt_shell> report_timing -delay min -from pad[0]
```

问题

问题 15. 输入外部延迟（input external delay）约束的值是否符合你的预期？

答：是。由上文 report_port 可知，输入外部延迟相对于 PCI_CLK 上升沿应为 2 ns，时序报告确认了这一点：

```
pt_shell> report_timing -delay min -from pad[0]
Startpoint: pad[0] (input port clocked by PCI_CLK)
Endpoint:   I_ORCA_TOP/I_PCI_CORE/d_out_i_bus_reg[0]
            (rising edge-triggered flip-flop clocked by PCI_CLK)
Path Group: PCI_CLK
Path Type:  min
...
clock PCI_CLK (rise edge)          0.000    0.000
clock network delay (propagated)    0.000    0.000
input external delay                2.000    2.000 r
pad[0] (inout)                     0.000    2.000 r
```

2.2.3 任务 3：分析输出延迟约束的时序报告

Task 3. Analyze a Timing Report For Output Delay Constraint

1. 生成施加于端口 pad[0] 的输出延迟约束报告：

```
pt_shell> report_port -output_delay pad[0]
```

问题

问题 16. 该端口的最小与最大输出延迟约束分别是多少？

答：

```
pt_shell> report_port -output_delay pad[0]
Output Delay
Min    Max    Related    Related
Output Port Rise Fall Rise Fall Clock Pin
pad[0]    -1.00 -1.00  4.00  4.00  PCI_CLK  --
```

最小输出延迟约束为 -1.00 ns ，最大输出延迟约束为 4.00 ns (rise/fall 相同)。

问题 17. 负的最小输出延迟约束将如何施加于该端口 (即它会施加正向还是负向的 hold 要求)?

答: 课堂讲义指出: 负的 hold output delay 约束会施加正向 hold 要求。

问题 18. 约束该端口的外部终点时钟名称是什么?

答: 端口 pad[0] 相对于 **PCI_CLK** 施加约束。

2. 生成一份以端口 pad[0] 为终点的 hold “简短 (short)” 时序报告:

```
pt_shell> report_timing -delay min -to pad[0] -path short
```

问题

问题 19. 描述该时序路径的起点 (即它是 input 端口还是内部触发器)。

答: 时序路径起点是内部 flip-flop:

```
pt_shell> report_timing -delay min -to pad[0]
Startpoint: I_ORCA_TOP/I_PCI_CORE/pad_out_buf_reg[0]
            (rising edge-triggered flip-flop clocked by PCI_CLK)
Endpoint:   pad[0] (output port clocked by PCI_CLK)
```

问题 20. 该时序路径的 path group 是否符合你的预期?

答: 是。path group 为 **PCI_CLK**，与外部捕获时钟名称一致。

问题 21. “data required time” 是否符合你的预期?

答: 是。捕获时钟边为 0 ns ，这对 hold 是合适的。hold 要求为正向 1 ns ，propagated 时钟网络延迟为 0 ns :

clock PCI_CLK (rise edge)	0.000	0.000
clock network delay (propagated)	0.000	0.000
output external delay	1.000	1.000
data required time		1.000

[可选步骤] 施加 `set_output_delay -min 1.0 -clock PCI_CLK pad[0]` 后:

```
pt_shell> report_port -output_delay pad[0]
Output Delay
Min    Max    Related    Related
Output Port Rise Fall Rise Fall Clock Pin
pad[0]    1.00  1.00  4.00  4.00  PCI_CLK  --

pt_shell> report_timing -to pad[0] -delay min -path short
```

```

...
clock network delay (propagated)      0.772    0.772
...
data arrival time                      3.445
clock PCI_CLK (rise edge)              0.000    0.000
clock network delay (propagated)      0.000    0.000
output external delay                  -1.000    -1.000
data required time                     -1.000
slack (MET)                            4.445

```

由上可见：对 output delay 使用正向 hold 约束会增大正向 slack。这证实：对 output delay 指定负 hold 约束，实际上是在 output 端口上指定 hold 要求。

3. [可选] 施加下列约束，为 pad[0] 施加正向 hold 输出延迟约束，然后重新执行本任务中的步骤以观察影响：

```
pt_shell> set_output_delay -min 1.0 -clock PCI_CLK pad[0]
```

4. 退出 PrimeTime:

```
pt_shell> quit
```

至此 Lab 2 完成。请返回课堂讲义。

Chapter 3

Lab 3: 生成报表

Lab 3: Generating Reports

学习目标

完成本实验后，你应该能够：

- 为 ORCA 中的违例生成汇总报告；
- 分析 setup 与 hold 时序报告；
- 使用正确的时序报告开关（switches）；
- 识别半时钟周期（half clock cycle）路径。

实验时长

约 45 分钟。

3.1 概述

Overview

本实验大致步骤如下：

1. 恢复 PrimeTime 会话；
2. 生成各类汇总报告；
3. 生成并分析时序报告。

3.1.1 相关文件与目录

Relevant Files and Directories

本实验全部文件位于主目录下的 lab3_reports 目录中（该目录为指向 lab2_constraints 的符号链接）。

路径 / 文件	说明
lab3_reports/	当前工作目录
orca_savesession/	用于恢复实验的会话目录
.synopsys_pt.setup	自动读取的 PT setup 文件

3.2 操作说明

Instructions

3.2.1 任务 1: 为 Lab 3 配置 PrimeTime

Task 1. Setup PrimeTime For Lab 3

1. 从 lab3_reports 目录（指向 lab2_constraints 的 workshop 实验目录）启动 PrimeTime，并使用 orca_savesession 目录恢复会话：

```
unix% cd lab3_reports
unix% pt_shell
pt_shell> restore_session orca_savesession
```

2. 查找控制许多报告有效数字位数的变量，并将其设为 4 位有效数字。[提示：aa significant]

```
pt_shell> set report_default_significant_digits 4
```

问题

问题 1. 该变量的默认值是多少？[提示：man page]

答：

```
pt_shell> set_app_var report_default_significant_digits -default
> report_default_significant_digits = "2"

pt_shell> set report_default_significant_digits 4
```

默认值为 2 位有效数字。

3.2.2 任务 2: 生成汇总报告

Task 2. Generate Summary Reports

从 Lab 1 可知 ORCA 中存在 setup 违例。

1. 通过生成合适的汇总报告回答下列问题：

问题

问题 2. 找出 slack 最差的 5 个 setup 违例，需给出端点名称与 slack。

答：下列命令将列出按 slack 排序的全部 setup 违例。使用 page mode 可从长报告中退出——

所需信息仅为前 5 个违例。

```
pt_shell> report_analysis -status violated -check setup -nosplit
```

Constrained Pin	Related Pin	Clock	Type	Slack
I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]/D	CP(rise)	SYS_CLK	setup	-0.9872
I_ORCA_TOP/I_BLENDER/s4_op1_reg[31]/D	CP(rise)	SYS_CLK	setup	-0.8410
I_ORCA_TOP/I_BLENDER/s4_op2_reg[30]/D	CP(rise)	SYS_CLK	setup	-0.8305
I_ORCA_TOP/I_BLENDER/s4_op1_reg[15]/D	CP(rise)	SYS_CLK	setup	-0.6918
I_ORCA_TOP/I_BLENDER/s4_op1_reg[30]/D	CP(rise)	SYS_CLK	setup	-0.6843

问题 3. 列出存在违例 setup 时序路径的 2 个时钟域，以及存在违例 hold 时序路径的 5 个时钟域（ORCA 共有 6 个时钟域）。

答：

```
pt_shell> report_constraint -all_violators -max_delay -min_delay
max_delay/setup ('PCI_CLK' group)
max_delay/setup ('SYS_CLK' group)
min_delay/hold ('PCI_CLK' group)
min_delay/hold ('SDRAM_CLK' group)
min_delay/hold ('SD_DDR_CLK' group)
min_delay/hold ('SYS_2x_CLK' group)
min_delay/hold ('SYS_CLK' group)
```

违例 setup 的 2 个时钟域：**PCI_CLK**、**SYS_CLK**。违例 hold 的 5 个时钟域：**PCI_CLK**、**SDRAM_CLK**、**SD_DDR_CLK**、**SYS_2x_CLK**、**SYS_CLK**。

问题 4. 分别有多少 hold 违例位于 input 路径、output 路径，以及 register-to-register 路径？

答：

```
pt_shell> report_global_timing
```

Hold violations					
	Total	reg->reg	in->reg	reg->out	in->out
WNS	-0.4375	-0.1420	-0.2363	-0.1281	-0.4375
TNS	-12.6317	-3.5616	-2.5768	-0.5954	-5.8979
NUM	100	59	18	7	16

input 路径 18 个，output 路径 7 个，register-to-register 路径 59 个。

2. 为以 output 端口 sd_DQ[0] 至 sd_DQ[15] 为终点的 16 位总线，生成各 bit 的 setup 最差 slack 报告（这些 output 端口均由单一时钟 SD_DDR_CLK 约束）。

问题

问题 5. 列出 slack 裕量最大（最好 slack）的端点。

答：output 端口 sd_DQ[0] 的 slack 最好，为 **1.5994 ns**。

一般情况下，下列命令对每个端点仅生成一条报告（`nworst` 默认为 1，且所有 output 端口由同一时钟约束）。但增大 `max_paths` 会隐式使用 `slack_lesser_than 0`，而所有到达该端点

的 slack 均为正值，因此须将 `slack_lesser_than` 改为较大正数（此处为 100）：

```
pt_shell> report_timing -path_type end -max_paths 16 \
    -slack_lesser_than 100 -to sd_DQ*
Endpoint          Path Delay Path Required CRP      Slack
sd_DQ[14] (inout)  8.4133 f    9.6649      0.2418  1.4933
...
sd_DQ[0] (inout)   8.0654 f    9.6649      0.0000  1.5994
```

3. 生成设计质量的高层概览：

问题

问题 6. 哪个 clock group 的违例路径数量最多？

答：

```
pt_shell> report_qor -only violated
```

SDRAM_CLK 有 **30** 条违例路径，为最多。

3.2.3 任务 3：分析 Setup 与 Hold 时序报告

Task 3. Analyze Timing Reports for Setup and Hold

1. 打开 page mode:

```
pt_shell> page_on
```

2. 执行下列命令，为 PCI_CLK 生成时序报告：

```
pt_shell> report_timing -group PCI_CLK
```

问题

问题 7. 该时序路径是满足（meet）还是违例（violate）时序？

答：违例时序，slack 为 -1.157 ns 。

问题 8. 该时序路径属于哪一类——内部 flip-flop 到 flip-flop、input，还是 output 时序路径？

答：这是一条 **output** 时序路径，终点为 output 端口 pad[1]。

3. 为同一 clock group PCI_CLK 生成 hold 时序报告：

```
pt_shell> report_timing -group PCI_CLK -delay min
```

问题

问题 9. 该时序路径属于哪一类——内部 flip-flop 到 flip-flop、input，还是 output 时序路径？

答：这是一条 内部时序路径。终点外观类似 flip-flop，实为 RAM 时序模型中的众多时序弧

之一。

问题 10. 该时序路径的数据通路上有多少个单元 (cell)?

答: 数据通路上仅有 1 个单元——起点 flip-flop。该时序路径由起点 flip-flop 直接连到终点 flip-flop。

问题 11. 起点 flip-flop 的 CP 到 Q pin 使用的是上升沿 (rise) 单元延迟。请给出一个可能的原因: 为何这比使用该 flip-flop 更快的下降沿 (fall) 延迟会导致更差的 hold slack?

答: 通常 fall 延迟比 rise 延迟更快, 会对 hold 产生更差的 slack! 本例的例外是: 若终点 flip-flop 数据 pin 的 fall 跳变导致更小的 library hold time 要求——这正是本例的情况。

下一步将继续探索并确认你对上述问题的回答。

问题

问题 12. 要确认上述问题的答案, 你还需要哪些额外信息?

答: 再生成一份以终点下降沿跳变计算数据到达时间的时序报告, 并比较两份报告。由此可确认: 终点 flip-flop 数据 pin 的下降跳变确实使 library hold time 更小, 从而 slack 更好。(下一步实验将探索这一点。)

4. 为同一条时序路径再生成一份 hold 报告, 但终点使用下降沿跳变 (而非上升沿)。请复制粘贴 pin 名以避免拼写错误。可参考 “timing reports” job aid 查找 report_timing 的合适开关。

问题

问题 13. 你使用了该报告中的哪些行来确认报告了正确的路径?

答:

```
pt_shell> report_timing -delay_type min_fall \
    -to I_ORCA_TOP/I_PCI_READ_FIFO/PCI_RFIFO_RAM/A1[1] \
    -from I_ORCA_TOP/I_PCI_READ_FIFO/count_int_reg[1]/CP
Startpoint: I_ORCA_TOP/I_PCI_READ_FIFO/count_int_reg[1]
             (rising edge-triggered flip-flop clocked by PCI_CLK)
Endpoint:   I_ORCA_TOP/I_PCI_READ_FIFO/PCI_RFIFO_RAM
             (rising edge-triggered flip-flop clocked by PCI_CLK)
Path Group: PCI_CLK
Path Type:  min
```

问题 14. 猜测是否正确——更快的 fall 延迟导致更快的数据到达时间, 但更小的 hold 时间要求, 从而得到更好的 slack?

答: 是! 数据到达时间更快 (1.139 ns 对 1.150 ns), 但 hold 时间要求更小 (0.211 ns 对 0.411 ns), 因此 slack 更好。Recall: hold time (以及 setup time) 是终点 flip-flop 数据 pin 跳变的函数。

data arrival time		1.139
library hold time	0.212	1.025
data required time		1.025
slack (MET)		0.114

3.2.4 任务 4: 使用正确的时序报告开关

Task 4. Apply the Correct Timing Report Switches

1. 通过在 PrimeTime 中实验与探索回答下列问题。可参考“timing reports”job aid 识别合适的命令与开关。

问题

问题 15. 写出为每个 path group 生成一条 setup 时序报告的命令。

答:

```
pt_shell> report_timing -group [get_path_group *]
```

问题 16. 写出为每个存在违例的 path group 生成一条 setup 时序报告的命令。

答:

```
pt_shell> report_timing -group [get_path_group *] -slack_lesser_than 0
# 交互式使用时, 可缩写命令名或开关, 或使用 Tab 补全:
pt_shell> report_timing -slack_less 0
```

问题 17. ORCA 中存在违例时序路径的两个 path group 名称是什么 (答案来自上一题的结果)?

答: 两个 path group 为 **PCI_CLK** 与 **SYS_CLK**。

问题 18. 写出为受 PCI_CLK 约束的任意 output 端口生成 setup 最差 slack 时序报告的命令。

答:

```
pt_shell> help all_*
pt_shell> all_outputs -help
pt_shell> report_timing -to [all_outputs -clock PCI_CLK]
# 或另一种等价写法:
pt_shell> report_timing -to [all_outputs] -group PCI_CLK
```

问题 19. ORCA 中有若干 latch; 写出识别这些 latch 数据 pin 的命令。

答:

```
pt_shell> all_registers -level_sensitive -data_pins
```

问题 20. 写出为 latched_clk_en_reg latch 的 D pin 生成 hold 时序报告的命令。

答: 请复制粘贴以避免拼写错误:

```
pt_shell> report_timing -delay min \
    -to I_ORCA_TOP/I_BLENDER*/latched_clk_en_reg/D
```

3.2.5 任务 5：识别半时钟周期路径

Task 5. Identify Half-Clock Cycle Paths

时钟 SDRAM_CLK 在 ORCA 中约束了许多半时钟周期路径（即从下降沿触发 flip-flop 到上升沿触发 flip-flop，或反之）。这些路径须仔细监控（例如 SDRAM_CLK 的占空比尚未明确定义，或用于分析时钟偏斜）。

1. 执行下列命令报告 SDRAM_CLK 的时钟周期，并据此回答问题：

```
pt_shell> report_clock SDRAM_CLK
```

问题

问题 21. 给定 waveform 列中第一个数为 SDRAM_CLK 的第一个上升沿、第二个数为下降沿——该时钟定义了多大的占空比？

SDRAM_CLK 上升沿在 **0 ns**，下降沿在 **3.75 ns**，周期为 **7.50 ns**。占空比为 **50%**。

问题 22. 对于由 SDRAM_CLK 上升沿约束到 SDRAM_CLK 下降沿的时序路径，描述 setup 时序报告中将使用的具体时钟边。

答：对于由 SDRAM_CLK 上升沿约束到下降沿的时序路径，setup 使用的时钟边为 **0 ns** 到 **3.75 ns**。

问题 23. 对于同一条时序路径，描述 hold 时序检查将使用的具体时钟边。

答：对于同一条时序路径，hold 使用的时钟边为 **7.5 ns** 到 **3.75 ns**。

```
# 半时钟周期路径报告命令示例（反斜杠为续行符）
# -delay_type min_max 将同时生成 setup 与 hold 各一份报告
report_timing -rise_from [get_clocks SDRAM_CLK] \
    -fall_to [get_clocks SDRAM_CLK] -delay_type min_max
report_timing -fall_from [get_clocks SDRAM_CLK] \
    -rise_to [get_clocks SDRAM_CLK] -delay_type min_max
```

2. 通过为 SDRAM_CLK 约束的半时钟周期时序路径生成合适的时序报告，确认下表中的信息：

Setup			Hold		
Launch	Capture	Slack	Launch	Capture	Slack
Fall	Rise 3.75 ns	0.680 ns	Fall	Rise 3.75 ns	3.558 ns
Fall	Rise 7.50 ns	0.635 ns	Fall	Rise 3.75 ns	3.514 ns

问题

问题 24. 哪个开关可用于为每类半时钟周期时序路径生成最差的 10 条时序报告？

答：在上述命令中添加开关 **-max_paths 10**。

问题 25. 为何 PrimeTime 报告 “no constrained paths”？（提示——PrimeTime 使用的选项显示在 report_timing 命令之后。）

-max_paths 会隐式设置另一选项 **slack_lesser_than 0**；由于 slack 为正值，因此没有可报告的路径。

问题 26. 要报告最差的 10 条时序路径，还必须使用哪个额外选项？

答：由于不存在违例路径可报告，须添加选项：`slack_lesser_than 100`。

3. 退出 PrimeTime:

```
pt_shell> quit
```

至此 Lab 3 完成。Day-1 课程结束。

Chapter 4

Lab 4: 约束多时钟

Lab 4: Constraining Multiple Clocks

学习目标

完成本实验后，你应该能够：

- 运用课堂讲授的命令收集设计中时钟的相关信息；
- 使用 GUI 从另一个视角查看设计时钟及其关系；
- 识别并报告虚假违例（false violation）；
- 确认分析结果。

实验时长

约 45 分钟。

4.1 概述

Overview

本实验大致步骤如下：

1. 在 shell 中熟悉设计时钟；
2. 在 GUI 中查看时钟关系；
3. 使用 GUI 深入探索时序路径细节；
4. 报告虚假违例；
5. 重新执行运行脚本以减少违例。

4.1.1 相关文件与目录

Relevant Files and Directories

本实验全部文件位于主目录下的 **lab4_clocks** 目录中。

路径 / 文件	说明
lab4_clocks/	当前工作目录
orca_savesession/	ORCA 初始保存会话
orca_savesession_violations/	含问题的 ORCA 保存会话
RUN.tcl	ORCA 运行脚本
.synopsys_pt.setup	PT setup 文件
scripts/	
orca_pt_variables.tcl	变量脚本

4.2 操作说明

Instructions

4.2.1 任务 1：熟悉设计时钟

Task 1. Get to Know the Design Clocks

1. 确认当前目录为 **lab4_clocks**。
2. 启动 PrimeTime (**pt_shell**)。恢复保存在 **./orca_savesession** 中的会话。可键入少量字符后按 **Tab** 键利用命令与文件名补全。
3. 运用课堂讲授的命令回答下列问题。可借助 Job Aid 「Clocks and More」回忆具体命令。

问题

问题 1. 设计中有多少个时钟？其中有多少个是 generated clock？

答：可用 **report_clock** 或下列命令获取：

```
pt_shell> sizeof_collection [all_clocks]
6
pt_shell> sizeof_collection [get_generated_clocks *]
3
```

设计中共有 **6** 个时钟，其中 **3** 个为 generated clock。

问题 2. 哪些输入端口定义了 master clock？

答：

```
pt_shell> rpt_clock_ports
Port Name  Direction  Clock Name  Is Generated
pclk       in         PCI_CLK     false
sys_clk    in         SYS_CLK     false
sdr_clk     in         SDRAM_CLK   false
sd_CK      out        SD_DDR_CLK  true
sd_CKn     out        SD_DDR_CLKn true
```

定义了 master clock 的输入端口为 **pclk**、**sys_clk**、**sdr_clk**。

问题 3. 哪些输出端口定义了 outgoing clock?

答: 由同一报告可知, 输出端口为 **sd_CK** 与 **sd_CKn**。

问题 4. 时钟是 propagated 还是 ideal?

答: 用 report_clock 可见, 设计中所有时钟均为 **ideal**。

问题 5. 哪 3 对时钟之间存在已约束的时序路径?

答:

```
pt_shell> check_timing -override clock_crossing -verbose
Information: Checking 'clock_crossing'.
Information: There are 4 clocks having domains interacting.
# all paths are false paths
# part of paths are false paths
From Clock      Crossing Clocks
PCI_CLK         SYS_CLK*
SDRAM_CLK       SD_DDR_CLK#, SYS_CLK*
SYS_2x_CLK      SDRAM_CLK*, SYS_CLK
SYS_CLK         PCI_CLK*, SDRAM_CLK*, SYS_2x_CLK
```

存在已约束时序路径的 3 对时钟为: **SDRAM_CLK** 与 **SYS_CLK**; **SYS_2x_CLK** 与 **SYS_CLK**; **SYS_CLK** 与 **PCI_CLK** (以及 **SDRAM_CLK**、**SYS_2x_CLK**)。

4.2.2 任务 2: 使用 GUI 报告时钟关系

Task 2. Use the GUI to Report Clock Relationships

若设计中时钟较多, GUI 可简化管理时钟之间关系的任务。

1. 执行下列命令启动 GUI:

```
pt_shell> start_gui
```

注意

原始 pt_shell 会话仍在终端窗口中运行。可保持 GUI 打开, 并根据任务需要选择 shell 或 GUI 界面。

2. 查看时钟域交叉 (clock domain crossings): 从下拉菜单打开 **Clock > Clock Analyzer** 中的「clock domain matrix」。打开的 Clock Analyzer 窗口 (必要时点击时钟左侧的加号展开) 应与 check_timing 报告设计内时钟交叉时的信息一致。将鼠标悬停在矩阵方块上可查看存在何种 false path。与 check_timing -override clock_crossing -verbose 的文本输出相比, 图形化矩阵有时更易于理解。

窗口左侧列出每个 master clock 以及由该 master clock 派生的 generated clock。

问题

问题 6. SYS_2x_CLK 的 master clock 是什么？

答：SYS_CLK。

问题 7. SYS_2x_CLK 定义在哪个 pin/port（其「source」）上？（提示：可能需要拖开时钟矩阵窗口以露出更多时钟信息列。）

I_CLOCK_GEN/I_CLKMUL/CLK_2x。（可能需要拖开矩阵窗口才能看到 source pin。）

问题 8. SYS_2x_CLK 的 master clock 定义在哪个 pin/port 上？

答：sys_clk。

- 进一步探索：显示 SYS_2x_CLK 的时钟原理图——选中该时钟，然后右键 **Schematic of Selected Clocks**。展开名为 I_CLOCK_GEN/U20 的 MUX 的 fanin（提示：可用 **Select > By Name** 定位/高亮 U20）：双击输入 stub，如下图所示，继续双击直至 fanin 耗尽（例如已到达输入端口）。

注意

图例说明：[ce] 表示 fully constrained paths；Launch：表示启动时钟；矩阵中各时钟块显示周期与波形信息。在原理图中双击输入 pin stub 可继续向上展开 fanin。

问题

问题 9. 哪个端口连接到 MUX I_CLOCK_GEN/U20 的 select pin？

答：test_mode。

问题 10. 查看原理图是否有助于理解测试模式下的时钟方案？

答：是——两个时钟进入 MUX：一个来自时钟发生器，一个直接来自端口。test_mode 端口控制 select 线，使测试模式下可旁路时钟发生器，让设计直接由端口驱动。

- 用抽象时钟图（abstract clock graph）探索时钟关系：关闭原理图窗口，在 TopLevel 窗口中选择 **Clock > Clock Graph for All Clocks**。必要时按 **F8** 显示工具栏，勾选工具栏中的各项后点击 **Apply**。
- 在 Abstract Clock Graph 工具栏中选择 **Mux** 并点击 **Apply**，找出一对 muxed clock。
- 在 Abstract Clock Graph 中定位实例 I_CLOCK_GEN/U10（mx02d1）。（提示：可用 **Select > By Name** 定位/高亮 U10。）

问题

问题 11. 哪些时钟驱动 I_CLOCK_GEN/U10？

答：SYS_CLK 与 SYS_2x_CLK。

- 从时钟图窗口「放大」到感兴趣的对象：选中 I_CLOCK_GEN/U10，然后 **Schematic > Schematic View**。

问题

问题 12. 哪个端口驱动 I_CLOCK_GEN/U10 的 select 线?

答: **power_save**。

8. 返回 Abstract Clock Graph。

问题

问题 13. 从 abstract clock graph 窗口, 能否打开并显示与 Clock Analyzer 中相同的 SYS_2x_CLK 时钟原理图? (在 SYS_2x_CLK 上右键查找相应选项。)

答: 可以。选中 SYS_2x_CLK (可能需要放大以仅选中该时钟), 右键选择 **Schematic for Selected Clocks**。可能需要双击展开输入或输出 stub 才能得到与 Clock Analyzer 中完全相同的原理图。

9. 点击 Clock Analyzer 窗口右上角的“X”关闭该窗口。

10. 点击 Clock Schematic 与 Clock Analyzer 窗口右上角的“X”关闭这些窗口。

4.2.3 任务 3: 使用 GUI 探索时序路径细节

Task 3. Use the GUI to Explore Detail of Timing Paths

研究启动时钟与捕获时钟之间的路径——本任务将查看由 SYS_CLK 时钟的启动与捕获路径上的网络延迟 (network latency)。

1. 在 shell 中 (GUI 背后的终端窗口仍保持打开) 执行下列命令, 在检查路径之前将所有时钟设为 propagated, 以便 PrimeTime 计算时钟网络延迟 (约需一分钟)。令 PrimeTime 保存所有 pin 的 arrival time (这正是你将要检查的内容), 然后定义待检查的时序路径集合:

```
set_propagated_clock [all_clocks]
set timing_save_pin_arrival_and_slack true
update_timing
set my_paths [get_timing_paths -max 10 -group SYS_CLK \
    -path full_clock_expanded]
```

2. 从下拉菜单 **Timing > Path Analyzer** 输入你的违例路径集合。输入时序路径集合并点击 **Apply**。
3. 为这 10 条时序路径生成直方图: 在 Path Analyzer 中右键点击 **ALL**, 选择 **Create Histogram**, 然后 **Apply**; 在弹出的 Categories Table/Histogram 对话框中点击 **OK**。
4. 从直方图中对选定的路径启动 Path Inspector: 左键点击最左侧 (最差 slack) 的柱条以选中该组路径; 再选中最差路径, 点击 **Inspector**。
5. 在 Path Inspector 中检查时钟重汇聚悲观 (clock reconvergent pessimism, CRP): 在 data required 与 data arrival 段中向下滚动找到 CRP, 再横向滚动查看 CRP 占延迟的百分比。

问题

问题 14. capture delay 中有百分之多少来自 CRP?

答: **4.18%**。

问题 15. 该百分比是否对所有设计都具有代表性？

答：否——该数值取决于具体设计与具体路径。

6. 点击 Path Inspector 窗口底部的 **Schematic** 标签查看路径原理图。

7. 在原理图窗口中找到 CRP 点——即启动路径与捕获路径分叉前的最后一个 pin。

注意

鼠标手势（gestures/strokes）可简化缩放：按住中键并垂直拖动光标可「全屏缩放」；沿对象对角向上拖动可放大，向下拖动可缩小。

8. 要查看该 pin 上的 arrival time，必要时须先「展开」该 pin 的 buffer（默认 PrimeTime 会将 buffer 树折叠为单个符号）。

9. 选中寄存器前一级 buffer 的输出 pin，然后选择 **View > Property**，查看 arrival time 及其他感兴趣的属性。

问题

问题 16. 该 buffer 输出 pin 的 arrival window 有多宽？

答：为 $2.86939 - 2.40107 \approx 0.468$ ns。

问题 17. 这是否与先前在 Path Inspector 的 data arrival / data required 段中看到的一致？

答：是。

10. 检查路径波形：点击 Path Inspector 窗口底部的 **Waveform** 标签。

问题

问题 18. 在波形窗口中右键点击可添加什么内容？

答：可添加输入 pin 与输出 pin，从而查看整条路径的细节。

11. 关闭 GUI 但保持原始 pt_shell 会话在终端中继续运行：

```
pt_shell> stop_gui
```

或在主 GUI 窗口选择 **File > Close GUI**。

12. 退出 PrimeTime。

4.2.4 任务 4：报告虚假违例

Task 4. Report a False Violation

1. 启动 PrimeTime 并恢复保存在 orca_savesession_violations 中的会话。

2. 确定 ORCA 中时序违例的数量与类型：

```
pt_shell> report_analysis_coverage
```

问题

问题 19. ORCA 有多少个违例？分别是什么类型？

答：共有 23 个 setup 违例、53 个 hold 违例、32 个 out_setup 违例。

3. 为 out_setup 时序检查中最差 slack 生成一份「简短 (short)」时序报告。

问题

问题 20. 如何找出 out_setup 中端点 port 的最差 slack？（可借助 Job Aid 「Timing Reports」回忆两个合适的开关。）

答：

```
pt_shell> report_timing -path short -to [all_outputs]
```

或：

```
pt_shell> page_on
pt_shell> report_analysis_coverage -status violated -check out_setup
Type of Check  Total  Met      Violated
out_setup      75    43 (57%)  32 (43%)
```

问题 21. 该违例涉及哪些时钟（launch 与 capture）？

答：

```
pt_shell> report_timing -to sd_DQ[6] -path short
Startpoint: sdr_clk (clock source 'SDRAM_CLK')
Endpoint:   sd_DQ[6] (output port clocked by SD_DDR_CLK)
Path Group: SD_DDR_CLK
Path Type:  max
```

Launch 时钟为 SDRAM_CLK，capture 时钟为 SD_DDR_CLK。

注意

由任务 1 可知，SD_DDR_CLK 是定义在输出端口上的 generated clock。定义 outgoing clock 的目的是让 PrimeTime 计算该时钟的 source latency，并将其纳入 data required time。

4. 查看上一步时序报告的数据 required time 段，注意其中未报告 clock latency。用下列命令确认：

```
# 该报告将无输出，因为 PrimeTime 尚未为 SD_DDR_CLK 计算 source latency
pt_shell> report_clock -skew SD_DDR_CLK
```

问题

问题 22. 为何 PrimeTime 未为 outgoing clock SD_DDR_CLK 计算 source latency？

答：必须为 propagated clock（尤其是 master clock），PrimeTime 才能为 generated clock 计算

source latency。本设计中所有时钟均为 ideal。

注意

与设计师沟通后发现存在误解：设计师期望你打开一个变量，使所有时钟均为 propagated！

5. 有一个变量可使所有时钟变为 propagated。用 Tcl 过程 aa 帮助定位该变量：

```
pt_shell> aa propagate
```

问题

问题 23. 该变量的名称是什么？

答： **timing_all_clocks_propagated**。

```
pt_shell> aa propagate
...
timing_all_clocks_propagated = "false"
```

问题 24. 根据 man page，该变量将做什么？

答：将该变量设为 true 后，之后创建的所有时钟都将以 propagated clock 方式创建。（下一任务中将实际设置该变量。）

6. 查阅 check_timing 的 man page，找出会标记所有 ideal clock 的附加检查名称。下列命令在带滚动条的弹出窗口中打开 man page，便于阅读长文档：

```
pt_shell> vman check_timing
```

注意

上述命令为 .synopsys_pt.setup 中定义的别名，使用 SolvNet Doc Id 014947 上的 view 命令。若环境中未安装或未提供 Tk 包的主可执行文件 wish，别名 vman 将不可用。

问题

问题 25. 如何修改 check_timing 以添加检查，验证所有时钟均为 propagated？

答：附加检查名为 **ideal_clocks**。可用 lappend 将其加入变量 timing_check_defaults，使 check_timing 自动执行该检查（下一任务中将在 ./scripts/orca_pt_variables.tcl 中完成）：

```
lappend timing_check_defaults ideal_clocks
set timing_all_clocks_propagated true
```

任务 5 步骤 4 验证要点：

- out_setup 违例应已减少；
- report_clock 显示所有时钟均为 propagated；
- check_timing 执行默认检查及 ideal_clocks 检查；

- `report_clock -skew SD_DDR_CLK` 显示 source latency 已计算;
- 到 `sd_DQ[6]` 的时序报告包含 clock network delay (propagated), slack 由违例变为 **MET** (1.5143 ns)。

7. 退出 PrimeTime。

4.2.5 任务 5: 重新执行运行脚本以减少违例

Task 5. Re-Execute the Run Script to Reduce Violation

1. 已提供文件 `./scripts/orca_pt_variables.tcl`, 它将完成以下两件事:

- 在 `check_timing` 默认检查之外, 增加会标记 ideal clock 的检查;
- 所有创建的时钟均以 propagated clock 方式创建。

2. 从 `lab4_clocks` Unix 目录执行运行脚本 `./RUN.tcl`, 并将结果记录到日志文件 `run.log`:

```
unix> pt_shell -f ./RUN.tcl | tee -i run.log
```

3. 启动 PrimeTime 并恢复新保存在 `./orca_savesession` 中的会话。

4. 用适当命令确认下列信息:

- `out_setup` 违例已减少;
- 所有时钟均为 propagated;
- 执行 `check_timing`, 确认除默认检查外还执行 ideal clock 检查;
- 现已为 `SD_DDR_CLK` 计算 source latency;
- 到 `sd_DQ[3]` 的时序报告包含该计算得到的 source latency。

注意

会出现额外违例 (更多 setup 违例以及 out_hold 违例), 可忽略。

5. 退出 PrimeTime。

至此 Lab 4 完成。请返回课堂讲义。

Chapter 5

Lab 5: 额外约束

Lab 5: Additional Constraints

学习目标

完成本实验后，你应该能够：

- 应用用户指定的 annotated delay 探索 latch 的 time borrowing（时间借用）；
- 调试时钟路径上 non-unate 单元相关的 **PTE-070** 信息消息。

实验时长

约 30 分钟。

5.1 概述

Overview

本实验涵盖两类额外约束场景：调试时钟网络上的 non-unate 路径警告，以及借助 annotated delay 探索 latch time borrowing。

5.1.1 相关文件与目录

Relevant Files and Directories

本实验全部文件位于主目录下的 `lab5_additional` 目录中。

路径 / 文件	说明
<code>lab5_additional/</code>	当前工作目录
<code>orca_savesession/</code>	ORCA 保存会话
<code>RUN.tcl</code>	创建 <code>orca_savesession</code> 的脚本
<code>logs/</code>	运行脚本产生的日志文件
<code>.synopsys_pt.setup</code>	PT setup 文件

5.2 操作说明

Instructions

5.2.1 任务 1：调试 PTE-070 信息消息

Task 1. Debug PTE-070 Information Messages

1. 从 lab5_additional Unix 目录启动 PrimeTime。恢复保存在 ./orca_savesession 中的会话。
2. 下列为时钟网络上 non-unate 路径的完整消息。下一步将要求你生成经过该 pin 的时序报告。为便于复制粘贴、避免拼写错误，可在另一终端窗口的日志文件中查找该消息，或在 PrimeTime 内用 Unix 命令 grep:

```
# 来自 ./logs/run.Log
Information: A non-unate path in clock network detected.
Propagating both inverting and noninverting senses of clock
"SDRAM_CLK" from pin
'I_ORCA_TOP/I_SDRAM_IF/sd_mux_dq_out_0/Z'. (PTE-070)
pt_shell> sh grep -A 1 -B 1 PTE-070 logs/run.log
```

注意

命令 sh（或 exec）允许在 PrimeTime shell 内执行 Unix 命令。

3. 为 setup 生成经过上述 pin 的时序报告，并回答下列问题。下列别名已在 PrimeTime setup 文件中创建，将借助 SolvNet Doc Id 014947 上的 view 工具在带滚动条的弹出窗口中生成时序报告：

```
pt_shell> vrt -through <through pin>
```

问题

问题 1. 你用时序报告中哪些行来确认这是 setup 报告，且路径起点为时钟 SDRAM_CLK 的 source?

答：

```
pt_shell> report_timing -through \
    I_ORCA_TOP/I_SDRAM_IF/sd_mux_dq_out_0/Z
Startpoint: sdr_clk (clock source 'SDRAM_CLK')
Endpoint:   sd_DQ[0] (output port clocked by SD_DDR_CLK)
Path Group: SD_DDR_CLK
Path Type:  max
```

Path Type: max 表示 setup; Startpoint: sdr_clk (clock source 'SDRAM_CLK') 表明起点为 SDRAM_CLK 的 source。

问题 2. 该时序报告如何确认警告中的 pin 位于 data path（即时钟 source 被当作数据路径使用并约束），而非 clock path?

答：若未生成报告（“path is unconstrained”），则该 pin 位于 clock path。由于生成了时序报告，

该 pin 位于 **data path**。

问题 3. 经过上述 pin 传播的是哪种 sense (positive unate 还是 negative unate)? 在时序报告中查找标示该 pin 的小箭头。

答: 经过该 pin 的是 **positive unate** 时序弧 (fall to fall)。数据通路中的相关行:

```
I_ORCA_TOP/I_SDRAM_IF/bufd7G5B2136/Z (bufd7) 0.1634 & 4.8748 f
I_ORCA_TOP/I_SDRAM_IF/sd_mux_dq_out_0/Z (mx02d4) <- 0.5431 & 5.4179 f
```

箭头 <- 表示 **negative unate**; 无箭头表示 **positive unate**。

- 再生成至少一份时序报告, 展示经过该 pin 的 **negative unate** 时序弧。

问题

问题 4. 你用时序报告中哪些行来确认这是 setup 报告、路径起点为 SDRAM_CLK 的 source, 且该 pin 的时序弧为 **negative unate**?

答:

```
pt_shell> report_timing -rise_through \
    I_ORCA_TOP/I_SDRAM_IF/sd_mux_dq_out_0/Z
Startpoint: sdr_clk (clock source 'SDRAM_CLK')
Endpoint:   sd_DQ[0] (output port clocked by SD_DDR_CLK)
Path Group: SD_DDR_CLK
Path Type:  max
I_ORCA_TOP/I_SDRAM_IF/bufd7G5B2136/Z (bufd7) 0.1634 & 4.8748 f
I_ORCA_TOP/I_SDRAM_IF/sd_mux_dq_out_0/Z (mx02d4) <- 0.4928 & 5.3676 r
```

Path Type: max 与起点确认 setup 及 SDRAM_CLK source; 箭头 <- 表示该 pin 为 **negative unate** 弧。

问题 5. 说明为何对这些时序路径可忽略 (并抑制) 该警告。

答: 该消息表示传播时钟经过该 MUX 时将同时使用反相与非反相 sense——这是默认行为。但由于该时钟被当作数据使用, PrimeTime 实际上会传播两种 sense (positive 与 negative unate), 即使在较旧版本的 PrimeTime 中也是如此。这正是期望的行为, 因此该信息消息可忽略并抑制。

- 不要退出 PrimeTime。

5.2.2 任务 2: 探索 Time Borrow 与 Latch

Task 2. Explore Time Borrow and Latches

本设计中只有一个 latch。

- 用下列命令找到它 (可键入少量字符后按 **Tab** 键补全):

```
pt_shell> all_registers -level_sensitive
pt_shell> !! -clock_pins
pt_shell> all_registers -level_sensitive -data_pins
```

问题

问题 6. 该 latch 的 clock pin 名称是什么？

答：

```
pt_shell> all_registers -level_sensitive -clock_pins
{"I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/EN"}
```

问题 7. 三个 data pin 的名称分别是什么？

答：

```
pt_shell> all_registers -level_sensitive -data_pins
{"I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D",
 "I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/SC",
 "I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/SD"}
```

2. 从该 latch 生成 setup 时序报告（须明确以 clock pin 为起点，而非仅用单元名）。本实验将称该报告为「path segment #2」。

注意

该 latch 在 ORCA 设计中的功能是产生时钟门控信号，用于打开/关闭时钟 SYS_CLK。

问题

问题 8. 你如何知道该 latch 未从上一级经历 time borrow？

答：若该 latch 正在经历 time borrow，报告中会出现一行说明给予起点（即上一级）的时间。本报告中没有该行，故未经历 time borrow。

生成 path segment #2 的命令：

```
pt_shell> report_timing -from \
    I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/EN
```

3. 为上一级生成时序报告（本实验称为「path segment #1」）。以 latch 的 D 输入 pin 作为该时序路径的终点：

```
pt_shell> report_timing -to I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D
```

问题

问题 9. path segment #1 还能再花多少时间才会开始从 path segment #2 借用时间？

答：还可再花 **3.465 ns** 才会开始从 path segment #2 借用时间（等于正 slack）。

4. 通过标注 4 ns 的 net delay，强制 path segment #1 从 path segment #2 借用时间：

```
# 复制粘贴以避免 pin 名拼写错误
```

```
pt_shell> set_annotated_delay -net 4 \
    -to I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D
```

5. 再次生成 path segment #1 的时序报告（可用上下箭头在历史事件列表中滚动）。

问题

问题 10. path segment #1 从 path segment #2 借用了多少时间？

答： path segment #1 借用了 **0.5306 ns**（在时序报告的 data required time 段中注明）。

问题 11. path segment #1 的 slack 是多少？

答： slack 为 **0**。PrimeTime 恰好借用所需时间，使 slack 等于零。

6. 重新生成 path segment #2 的时序报告。在此之前须执行完整 timing update：

```
pt_shell> update_timing -full
```

注意

此时路径起点将为 latch 的 D pin（而非先前使用的 clock pin），因为你关注的是包含 time borrow 的时序路径。

```
pt_shell> report_timing -from \
    I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D
```

问题

问题 12. 给予 path segment #1 的时间是否符合你的预期？

答： 是——path segment #2 时序报告中给予起点的时间，与 path segment #1 时序报告中借用的 time borrow 一致。

```
pt_shell> report_timing -from I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D
pt_shell> report_timing -to I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D
```

7. 修改 latch 行为以实现 transparency（透明）：即当数据在时钟 opening 与 closing 边之间到达时，latch 保持透明。

```
set_app_var timing_enable_through_paths true
```

8. 重复到 latch D pin 的时序报告。注意即使 latch 处于透明状态，仍可将 D pin 指定为终点：

```
report_timing -to I_ORCA_TOP/I_BLENDER/latched_clk_en_reg/D
```

问题

问题 13. 起点（startpoint）是什么？

答： **I_ORCA_TOP/I_PARSER/blender_clk_en_reg/CP**。

9. 从刚识别的起点做时序报告：

```
report_timing -from I_ORCA_TOP/I_PARSER/blender_clk_en_reg/CP
```

问题

问题 14. 路径终点是什么？

答： I_ORCA_TOP/I_BLENDER/U794/A（gating check 终点）。

问题 15. transparency 的 open 与 close 边分别是多少？

答： transparency open 与 close 边分别为 6.6454 与 10.4433。

问题 16. 数据是否在两者之间到达？

答： 是——数据在 7.1759 到达，介于 open 与 close 边之间。

问题 17. 是否发生了 time borrowing？

答： 否，未发生 time borrowing。

问题 18. slack 是否为正？

答： 是，slack 为正（2.1398 ns）。

完整透明 latch 时序报告：

```
pt_shell> report_timing -from \
    I_ORCA_TOP/I_PARSER/blender_clk_en_reg/CP
Startpoint: I_ORCA_TOP/I_PARSER/blender_clk_en_reg
    (rising edge-triggered flip-flop clocked by SYS_CLK)
Endpoint: I_ORCA_TOP/I_BLENDER/U794
    (rising clock gating-check end-point clocked by SYS_CLK)
Path Group: **clock_gating_default**
Path Type: max
...
transparency open edge 6.6454
transparency close edge 10.4433
available borrow at through pin 3.2674
...
slack (MET) 2.1398
```

10. 退出 PrimeTime。

至此 Lab 5 完成。请返回课堂讲义。

Chapter 7

Lab 7: 基于路径的分析（PBA）

Lab 7: Path-Based Analysis

学习目标

完成本实验后，你应该能够：

- 使用 **path mode PBA** 与 **exhaustive PBA** 提高 PrimeTime 时序分析精度；
- 对违例路径执行交互式路径重算，并比较 GBA 与 PBA 结果；
- 生成并解读 PBA 汇总报告。

实验时长

约 30 分钟。

7.1 概述

Introduction

本实验将执行 exhaustive PBA（Exhaustive Path-Based Analysis），并在交互模式下对最坏违例路径进行 path mode PBA 重算。

7.1.1 相关文件与目录

Relevant Files and Directories

本实验全部文件位于主目录下的 **lab7_pba** 目录中。

路径 / 文件	说明
lab7_pba/	当前工作目录
.synopsys_pt.setup	自动读取的 PT setup 文件
orca_savesession/	已保存会话目录
RUN.tcl	用于重建 orca_savesession 的运行脚本

7.2 操作说明

Instructions

7.2.1 任务 1: 报告当前设计中的违例路径

Task 1. Report violating paths in the current design

本任务将报告在 **worst_slew propagation** 期间计算得到的最坏违例 slack。

1. 进入本实验工作目录:

```
unix% cd lab7_pba
```

2. 启动 PrimeTime, 并恢复名为 **orca_savesession** 的已保存会话:

```
unix% pt_shell
pt_shell> restore_session orca_savesession/
```

注意

- 如需重建该会话, 可使用:
`pt_shell -f RUN.tcl | tee -i run.log`
- 执行 **RUN.tcl** 时出现的任何 **PARA-124** 错误, 在本课程实验中可安全忽略。

3. 确定 **setup** 违例数量, 并找出到达某端点的最坏违例路径:

```
pt_shell> report_analysis_coverage -check setup \
-status violated
```

问题

问题 1. 有多少个 **setup** 检查处于 **violated** 状态?

答: 共有 **23** 个 **setup** 检查 **violated**。

问题 2. 最坏违例 **slack** 的大小是多少? 对应的端点 (endpoint) 是什么?

答: 最坏违例 **slack** 为 **-0.7737**, 端点为

I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]/D。

7.2.2 任务 2: 执行 **path mode PBA**

Task 2. Perform path mode PBA

本任务将交互式地重算已报告的最坏违例路径的 **slack**。

1. 使用 **PBA path** 模式确定最坏 **setup slack** 违例:

```
pt_shell> report_timing -to \
I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]/D -pba_mode path
```

或:

```
pt_shell> set path [get_timing_paths -to \
I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]/D -pba_mode path]
pt_shell> report_timing $path
```

问题

问题 3. `report_timing` 如何表明该路径已被重算？

答: **Path Type** 行后出现 (recalculated)。例如:

```
pt_shell> report_timing -to I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]/D \
-pba_mode path
...
Path Group: SYS_CLK
Path Type: max (recalculated)
```

问题 4. 交互式 path-based analysis (`-pba_mode path`) 之后, 最坏 slack 违例是多少?

答: 交互式 path-based analysis 之后最坏 slack 为 **-0.7711**。

问题 5. 与 GBA slack 相比, PBA slack 是改善、变差还是不变? 结果是否符合预期?

答: slack 有所改善, 符合预期。

(路径重算后 slack 只能保持不变或改善。)

注意

PBA 不会修改 PrimeTime 数据库, 因此不能再用 `report_analysis_coverage` 查看 PBA 结果。本实验中可使用带 `-pba_mode` 选项的 `report_timing` 或 `get_timing_paths`。

7.2.3 任务 3: 执行 exhaustive PBA

Task 3. Performing Exhaustive PBA

本任务将在 exhaustive PBA 模式下运行设计分析。

1. 找出时钟组 SYS_CLK 中前 10 个 setup 违例及其端点:

```
pt_shell> redirect -tee GBA.rpt { report_timing \
-path_summary -group SYS_CLK -max_paths 10 -nosplit }
```

2. 对设计中的 setup 违例执行 exhaustive PBA, 并生成汇总报告:

```
pt_shell> redirect -tee PBA.rpt { report_timing \
-path_summary -group SYS_CLK \
-max_paths 10 -nosplit -pba_mode exhaustive }
```

3. 比较 GBA.rpt 与 PBA.rpt:

问题

问题 6. GBA 与 PBA 各返回多少条违例路径?

答: GBA: 9 条违例路径; PBA exhaustive: 8 条违例路径。

```
pt_shell> sizeof_collection [get_timing_paths \
-slack_lesser_than 0 -group SYS_CLK -max_paths 10]
pt_shell> sizeof_collection [get_timing_paths \
-slack_lesser_than 0 -group SYS_CLK -max_paths 10 \
-pba_mode exhaustive]
```

问题

问题 7. exhaustive PBA 之后最坏违例的值是多少？对应的时序路径端点名是什么？

答：exhaustive PBA 之前最坏违例为 **-0.7737**，之后为 **-0.7711**；

端点均为 I_ORCA_TOP/I_BLENDER/s4_op2_reg[31]/D。

问题 8. 是否出现 **UITE-480** 警告？

答：没有 **UITE-480** 警告——exhaustive PBA 重算已完成。

7.2.4 任务 4: [可选] 生成 GBA 与 PBA 对比汇总报告

Task 4. [OPTIONAL Task] Generate GBA vs. PBA Summary Reports

本可选任务将分别用 GBA、PBA path 模式与 PBA exhaustive 模式生成全局、QoR 与约束报告以便对比。

```
redirect -tee GBASummary.rpt {
  report_global_timing
  report_qor
  report_constraint -all_violators
}
redirect -tee PBApath.rpt {
  report_global_timing -pba_mode path
  report_qor -pba_mode path
  report_constraint -all_violators -pba_mode path
}
redirect -tee PBAexhaust.rpt {
  report_global_timing -pba_mode exhaustive
  report_qor -pba_mode exhaustive
  report_constraint -all_violators -pba_mode exhaustive
}
```

至此 Lab 7 完成。Day-2 课程结束。

Chapter 8

Lab 8: SI 延迟分析

Lab 8: SI Delay Analysis

学习目标

完成本实验后，你应该能够：

- 修改常规 PrimeTime 运行脚本以启用 PrimeTime SI 串扰延迟（crosstalk delay）分析；
- 在 shell 与 GUI 中生成并解读 SI 报告，收集指导时序收敛的信息。

实验时长

约 45 分钟。

8.1 概述

Overview

本实验涵盖：修改运行脚本以启用 SI 延迟分析、在 shell 中解读 SI 时序报告，以及在 GUI 中查看串扰直方图。

8.1.1 相关文件与目录

Relevant Files and Directories

路径 / 文件	说明
ref/db/	工艺库
ref/design_data/	寄生参数与设计网表
ref/scripts/	支持脚本
ref/tcl_procs/	实用 Tcl 过程
lab8_si_delay/	本实验工作目录
RUN.tcl	常规 PrimeTime 运行脚本
SI-RUN.tcl	PrimeTime SI 运行脚本
.synopsys_pt.setup	Setup 文件
max_func_savesession/	常规 PrimeTime 已保存会话
rpt/	运行脚本生成的报告

8.2 操作说明

Instructions

8.2.1 任务 1: 执行 PrimeTime SI 运行脚本

Task 1. Execute a Run Script for PrimeTime SI

1. 回答下列问题:

问题

问题 1. 将“常规” PrimeTime 运行脚本修改为 PrimeTime SI 运行脚本，必须包含哪两个步骤？

答：必须包含的两个步骤：

```
# 在运行脚本开头设置该变量
set si_enable_analysis true
# 读寄生参数时保留耦合电容
read_parasitics -keep_coupling -format SBPF route_xtalk.sbpf
```

2. 进入目录 lab8_si_delay。可选：查看 SI 运行脚本 SI-RUN.tcl 及相关文件。

3. 执行 PrimeTime SI:

```
unix% pt_shell -f SI-RUN.tcl | tee -i sirun.log
```

4. 运行脚本完成后，启动 PrimeTime 并恢复运行期间创建的已保存会话。会话目录名含时间戳，多次运行会保存在不同 Unix 目录中：

```
unix% pt_shell
pt_shell> restore_session simax_func_savesession_<time_stamp>
```

注意

PrimeTime 支持命令、选项、变量与文件名补全：键入少量字符后按 **Tab** 键。

5. 使用 Tcl 过程 **aa** (always ask) 查找所有包含 **si_** 的命令/变量:

```
pt_shell> aa si_
```

注意

上述 Tcl 过程可在 SolvNet 查阅, Doc Id **012959**。

8.2.2 任务 2: 在 Shell 中生成并解读 SI 报告

Task 2. Generate and Interpret SI Reports in the Shell

1. 报告本设计中的违例总数, 并回答下列问题。

注意

常规 PrimeTime 已通过时序; 新出现的违例均与串扰相关。常规 PrimeTime 运行的会话保存在 **max_func_savesession** 目录中 (如有兴趣可对比)。

问题

问题 2. 本设计有多少个违例?

答:

```
pt_shell> report_analysis_coverage
```

共有 **14** 个违例, 且均为 setup 检查。

问题 3. 本设计中 setup 的最坏 slack 有多大?

答:

```
pt_shell> report_analysis_coverage -status violated
```

最坏 setup slack 约为 **-0.1422** (s4_op1_reg_30_/D)。

2. 生成到达最大负 slack 端点的时序报告 (建议复制粘贴端点名以免拼写错误):

```
pt_shell> report_timing -crosstalk -nets -transition \
-nosplit -to <endpoint>
```

注意

可选: 使用别名 **view** 在带滚动条的独立窗口中打开时序报告:

```
pt_shell> view report_timing -crosstalk -nets -transition \
-nosplit -to <endpoint>
```

view 对应的 Tcl 过程见 SolvNet Doc Id **014947**。若实验环境未安装 Tk 包中的 **wish** 可执行文件, **view** 别名将不可用。

问题

问题 4. 该路径上 delta delay 最大的网是哪条？最坏 delta delay 有多大？

答：该路径上最大 delta delay 为 **0.1024 ns**。

问题 5. 该 delta delay 如何表示（时间单位还是对 VDD 的比值）？

答：以时间单位表示；主库时间单位为 ns。

问题 6. delta delay 为正还是为负？是否符合你的预期？

答：delta delay 为正。合理——这是 setup 报告，数据路径上应考虑最坏情况下的串扰减速效应。

问题 7. 与该大 delta delay 对应的 stage delay 是多少？

答：stage delay 为单元 + 网延迟： $0.5522 + 0.1025 = 0.6547$ ns；

delta delay 与 stage delay 之比约为 **15%**。

问题 8. 报告中“&”符号的含义是什么？

答：表示延迟使用详细网寄生数据计算。符号含义（摘自 report_timing man 页）：

符号	反标类型
H	Hybrid annotation
*	SDF back-annotation
&	RC network back-annotation
\$	RC pi back-annotation
+	Lumped RC
<none>	Wire-load model 或 none

3. 对同一端点生成 hold 时序报告（也可用上箭头重复上一条命令并追加 -delay min）：

```
pt_shell> !! -delay min
```

问题

问题 9. delta delay 为正还是为负？是否符合你的预期？

答：delta delay 现为负，合理——这是 hold 报告，数据路径上应考虑最坏情况下的串扰加速效应。

注意

注意：该 hold 路径上网延迟可能为负，但应关注的 stage delay（有意义延迟数值）为正。

4. 参照 Job Aid，查找并执行合适命令，找出 delta delay 最大的 victim 网：

问题

问题 10. 本设计中最大的 delta delay 是多少？

答：违例时序路径上 setup 与 hold 的最大 delta delay 均为 **0.1738 ns**：

```
pt_shell> report_si_bottleneck -cost_type delta_delay
```

问题 11. 报告的网是 victim 还是 aggressor?

答: 这些是 **victim** 网。

问题 12. 报告的网是否位于违例时序路径上?

答: 是。默认 `-slack_lesser_than` 为 0, 因此报告的 victim 网均位于 setup 或 hold 违例路径上。

问题

问题 13. 该命令的哪个开关可让 delta delay 较大的时钟网也出现在报告中 (默认不包含)? (提示: 使用 man 页或 help。)

答: 开关为 `-include_clock_nets`:

键入下列内容 (含短横线) 后按 **Tab** 补全

```
pt_shell> report_si_bottleneck -
cost_type          max              min              nosplit
pre_commands       slack_lesser_than  include_clock_nets
max_nets           minimum_active_aggressors  post_commands
significant_digits
```

```
pt_shell> vman report_si_bottleneck
```

问题

问题 14. 如何减小这些网上的 delta delay, 从而消除或缓解时序违例?

(提示: 参阅 Job Aid 或 `report_si_bottleneck` 的 man 页。)

答: 使用 `set_coupling_separation` 增大 victim 与 aggressor 间距, 从而减小耦合电容。

5. 参照 Job Aid, 查找并执行合适命令, 找出 delta delay 与 stage delay 比值最大的 victim 网:

问题

问题 15. 该报告中的 cost 表示什么 (时间单位还是比值)?

答: cost 是 delta delay 与 stage delay 的比值:

```
pt_shell> report_si_bottleneck -cost_type delta_delay_ratio
```

问题 16. 报告的网是 victim 还是 aggressor?

答: 报告的网是 **victim**。

问题 17. 如何减小这些网上的 delta delay? (提示同上。)

答: 大比值通常由 victim 转换较慢 (弱 victim) 引起; 可通过加大 **victim** 驱动或在 victim 上插入 **buffer** 修复。

6. 参照 Job Aid, 查找并执行合适命令, 找出本设计中最强的 aggressor 网:

问题

问题 18. 该报告中的 cost 表示什么（时间单位还是对 VDD 的比值）？

答：cost 是相对 VDD 的比值——该 aggressor 网在所有相关违例 victim 网上引起的切换 bump 之和：

```
pt_shell> report_si_bottleneck -cost_type delay_bump_per_aggressor
```

问题 19. 报告的网是 victim 还是 aggressor？

答：这些是 **aggressor** 网。

问题 20. aggressor 网本身是否一定位于违例时序路径上？

答：不一定。只有 victim 网必须位于违例时序路径上（由 `-slack_lesser_than` 定义）。

问题 21. 如何减小这些 aggressor 网引起的 delta delay？（提示同上。）

答：缩小强 aggressor 驱动，或通过网隔离减小耦合电容。

7. 参照 Job Aid，查找并执行合适命令，报告最坏时钟偏斜及相关的串扰贡献：

问题

问题 22. 该报告是否表明时钟网屏蔽良好？

答：是。最坏时钟偏斜未被 delta delay 贡献恶化：

```
pt_shell> report_clock_timing -type skew -crosstalk -verbose
```

8.2.3 任务 3：在 GUI 中生成 SI 报告

Task 3. Generate SI Reports in the GUI

对于很长的时序报告，在报告中查找最大 delta delay 可能较困难；GUI 可简化该任务。

1. 启动 GUI：

```
pt_shell> start_gui
```

2. 从 **Crosstalk** 下拉菜单选择 **Delta Delay Histogram**，在对话框中点击 **OK**。

3. 在直方图中选择最右侧 bin，其中包含 setup 的最坏 delta delay。

4. 从 **Crosstalk** 菜单选择 **Accumulated Bump Voltage Histogram**，点击 **OK**。该视图按 victim 网显示累积 bump 电压（从最好到最坏）。

5. 探索 Crosstalk 菜单中的其他感兴趣选项。

6. 通过 **File > Close GUI** 或 shell 命令 `stop_gui` 关闭 GUI。

恭喜——Lab 8 完成！

Chapter 9

Lab 9: SI 噪声分析

Lab 9: SI Noise Analysis

学习目标

完成本实验后，你应该能够：

- 运行 PrimeTime SI 进行串扰噪声（crosstalk noise）分析；
- 在 shell 与 GUI 中生成并解读关键噪声报告。

实验时长

约 45 分钟。

9.1 概述

Overview

本实验涵盖：噪声分析运行脚本要点、shell 中的噪声报告解读，以及 GUI 中的噪声 bump 直方图分析。

9.1.1 相关文件与目录

Relevant Files and Directories

路径 / 文件	说明
ref/db/	工艺库
ref/design_data/	寄生参数与设计网表
ref/scripts/	支持脚本
ref/tcl_procs/	实用 Tcl 过程
lab9_si_noise/	本实验工作目录
run_noise.pt	PTSI 噪声运行脚本
.synopsys_pt.setup	Setup 文件
SE_PE_savesession/	含噪声分析的已保存会话
noise.log	SI 运行日志
rpt/	运行脚本生成的报告

9.2 操作说明

Instructions

9.2.1 任务 1：运行 PrimeTime SI 噪声分析

Task 1. Run PrimeTime SI for Noise Analysis

噪声运行脚本已预先执行并记录日志。

1. 进入目录 lab9_si_noise。
2. 打开运行脚本 run_noise.pt 并回答：

问题

问题 1. 列出该脚本中与噪声分析相关的重要步骤（至少三条）。

答：与噪声分析相关的步骤示例（任选其三即可）：

- 用变量 si_enable_analysis 开启 SI 分析（与 SI 延迟分析共用）；
- 使用包含噪声表征的库；
- 应用噪声 derating 因子缩放计算的噪声 bump 高度与宽度；
- 读入网寄生参数并保留耦合电容（与 SI 延迟分析共用）；
- 执行 update_noise 触发噪声更新（或执行 report_noise，其会同时更新并报告）。

问题

问题 2. 勾选噪声分析默认开启的选项（由 set_noise_parameters 控制）：

- ☐ Beyond rail analysis ☐ Between-the-rails analysis
☐ Noise propagation ☐ Timing windows are used to filter aggressors

答：默认开启：

- ☐ Beyond rail analysis ☒ Between-the-rails analysis

☐ Noise propagation ☒ Timing windows are used to filter aggressors

3. 打开日志文件 `noise.log` 并回答：

问题

问题 3. 哪条命令触发了 timing update（查找消息 **UITE-214**）？

答：check_timing 触发了 timing update。

问题 4. 该 timing update 是否包含串扰延迟分析？

答：是。可从 **XTALK-xxx** 消息看出。

问题 5. 哪条命令触发了 noise update（查找消息 **NOISE-010**）？

答：update_noise 触发了 noise update。

相关消息由应用变量 `si_noise_update_status_level` 开启。

问题 6. 根据课堂讲义，timing update 为噪声分析提供哪些信息？

答：使用 aggressor 网的到达窗口（**arrival windows**）进行滤波；仅窗口重叠的 aggressor 被定义为 active，并参与噪声 bump 计算。

9.2.2 任务 2：在 Shell 中解读噪声报告

Task 2. Interpret Reports for Noise in the Shell

1. 从 `lab9_si_noise` 目录启动 PrimeTime，并恢复 `SE_PE_savesession` 中的会话（善用 Tab 补全）：

```
unix% pt_shell
pt_shell> restore_session SE_PE_savesession
```

2. 执行下列命令，用 slack 类型 area 找出最坏噪声 slack：

```
pt_shell> report_noise -all_violators -slack_type area
pt_shell> report_noise -verbose -slack_type area
```

问题

问题 7. 本设计是否存在噪声违例？

答：没有噪声违例。

问题 8. 最坏噪声 slack（面积单位）是多少？相关区域是 `above_low` 还是 `below_high`？

答：最坏 slack 为 **153.8006**（area 单位），区域为 **above_low**。

问题 9. 对该“最坏”victim 网列出了多少个“有效（effective）”aggressor 网？

答：有 **1** 个有效 aggressor 网：n25422。

问题 10. 对该 victim 网，给出驱动单元 pin 名、负载 pin 名与网名。

答：victim 网名 n25449；驱动 pin T3_reg_190/QN；负载 pin U11405/A1。

问题 11. 解释报告中“Propagated”的含义，并说明该行为何可能为空（显示为破折号）。

答：“Propagated”行用于前级网噪声经驱动单元传播到本网的噪声。该行可能为空，因为未开启噪声传播（本分析即如此），和/或库不支持噪声传播（本实验库亦不支持）。

3. 对同一网执行下列命令，根据报告上部（噪声计算概要）回答问题：

```
# 从 report_noise 输出复制粘贴 pin 名以免拼写错误
pt_shell> report_noise_calculation -above_low -from <pin> -to <pin>
```

问题

问题 12. 解释 “Steady state resistance source: library set iv curve” 的含义。

答：表示计算噪声 bump 高度时，驱动输出稳态 I-V 特性所用的模型；其他选项例如 `set_steady_state_resistance` 设置的 “user set value”，或 PrimeTime 估计线性电阻时的 “estimation set value”。

问题 13. 是否对该网应用了噪声 derating 因子？是否与运行脚本一致？

答：是。高度与宽度缩放因子均为 **1.5**，与运行脚本一致。

问题 14. 解释 “noise effort threshold” 的含义。

答：噪声 bump 高度大于轨到轨电压的 **0.2** 倍（亦为默认值）的网，将使用基于 Arnoldi 阶约化的更复杂计算器。

4. 根据报告中部（噪声计算细节）回答：

问题

问题 15. 哪条命令可排除特定 aggressor 网，或将其定义为具有无限 timing window？（提示：参阅 Job Aid。）

`set_si_noise_analysis`。

5. 根据报告下部（噪声 slack 计算）回答：

问题

问题 16. 解释 “Constraint type: library immunity table” 的含义。

答：表示计算噪声约束（允许/所需噪声 bump 高度）所用的模型；其他选项例如 `set_noise_margin` 的 “user margin”，或库与用户均未提供约束时的 “none”。

问题 17. 噪声约束（库电压单位下的所需噪声 bump 高度）是多少？

答：噪声约束为 **0.6009**（库电压单位）。

问题 18. slack 的 “area” 与上一节噪声 bump 的 “area” 有何不同？

答：噪声 bump area 为 **%height × width**；
slack area 为 **height × width**。可用 shell 中的 `expr` 验证。

6. 查看所有寄存器数据 pin 上 slack 类型为 area_percent 的最差 5 条网：

```
pt_shell> report_noise -data_pins -nworst 5 -slack_type area_percent
```

问题

问题 19. 最差的触发器数据 pin 距离约束有多远（裕量很大还是很小）？

答：裕量较大：

above_low slack: **58.27%** (pin: T1_reg_120/SD)；

below_high slack: **INFINITY** (pin: T1_reg_120/SD)。

问题 20. 解释 slack 为 INFINITY 的含义。

答：若计算的噪声 bump 为零，则 slack 定义为 **INFINITY**。

可能原因：该网无有效 aggressor，或所有有效 aggressor 均被筛除。

- 查看本设计中寄存器时钟 pin 上的噪声：

```
pt_shell> report_noise -clock_pins
```

9.2.3 任务 3：在 GUI 中运行噪声分析

Task 3. Run Noise Analysis in the GUI

独立于 slack 与约束高度，识别具有较大 bump 高度的网可能很有意义；GUI 便于此类分析。

- 打开 GUI：

```
pt_shell> start_gui
```

- 通过工具栏按钮或 **Noise** 下拉菜单打开 **Accumulated Noise Bump Histogram**，在对话框中保留默认设置并点击 **OK**。

注意

该直方图按 bump 高度（非 slack）列出网或负载 pin（在对话框中选择），针对 below_high 区域（在对话框中选择）。

对本设计，**Composite Noise Bump Histogram** 结果相同；composite 噪声 bump 包含 propagated noise，而 accumulated 噪声 bump 不包含。

- 选择最坏 bin（最右侧柱）以显示 bump 高度最大的网名。

问题

问题 21. 有多少条网的噪声 bump 高度大于 0.5？

答：在 below_high 区域，有 **4** 条网的噪声 bump 高度大于 0.5：

```
pt_shell> report_noise_bumps -bump_greater_than 0.5
```

Pin Name	Net Name
U11361/A1	n25502
U13544/A2	C230
U11688/B1	AW0_199
U15062/A2	BS1_RDY

注意

SolvNet (Doc Id **009418**) 提供可在 shell 中报告相同信息的 Tcl 过程；实验环境已包含该过程。示例：

```
pt_shell> report_noise_bumps -bump_greater_than 0.5
```

4. 退出 PrimeTime。

恭喜——Lab 9 完成！

附录 A

Job Aid 速查卡

Job Aids

注意

原书 PDF 第 115–124 页为折叠式速查卡，版式以图示与命令列表为主。

其中第 115 页（*PrimeTime JOB AID: Timing Reports*）与第 116 页（*PT JOB AID: Run Scripts*）已从扫描页人工校对并译入下文；第 121 页（SI 串扰延迟微调）部分内容可辨认。

第 117–120、122–123 页 OCR 质量极差，正文以图示速查为主——若需完整版式请对照原版扫描 PDF。

第 124 页为空白页（*This page was intentionally left blank.*）。

A.1 PrimeTime JOB AID: Timing Reports

PrimeTime JOB AID: Timing Reports（PDF p.115）

A.1.1 实用 PrimeTime 命令

会话

```
restore_session <unix_dir>    ;# 恢复前先 save_session  
  
save_session <unix_dir>
```

帮助

```
report_timing -help  
  
help -verbose report_timing  
  
man report_timing  
  
help report*           ;# 查找命令
```

```
apropos clock ;           ;# 搜索命令数据库
```

环境与历史

```
printvar *sig*           ;# 查找变量

page_on                  ;# 别名

page_off                 ;# 别名

history

sh_list_key_bindings

set_app_var sh_line_editing_mode emacs
```

设计与库

```
list_libraries ; list_designs

report_lib <lib_name>     ;# 时间单位

remove_design -all

remove_lib -all
```

A.1.2 违例汇总报告

report_analysis_coverage

报告违例总数。

- -status_violated: 违例详情
- -check {setup hold}: 指定检查类型
- -sort_by slack: 默认排序
- -sort_by check_type: 按时钟组排序违例

report_constraint -all

报告全部约束。

report_global_timing

全局时序汇总。

report_qor -only_violated

仅报告违例路径的 QoR。

A.1.3 report_timing 开关

控制路径数量与筛选

- **-nworst**: 每个端点考虑的路径条数
- **-max_paths**: 每个设计生成的路径条数
- **-delay**: setup (max) 或 hold (min), 及端点数据跳变

max_rise、max_fall、min_rise、min_fall

- **-group**: 聚焦某 path group (如捕获时钟名)
- **-group [get_path_group *]**
- 路径过滤: **-to**、**-rise_to**、**-fall_to**、
-from、**-rise_from**、**-fall_from**、
-through、**-rise_through**、**-fall_through**、
-exclude、**-rise_exclude**、**-fall_exclude**、
-slack_lesser_than、**-slack_greater_than**

控制报告细节

- **-input_pins**: 显示输入 pin, 并分开网延迟与单元延迟
- **-nets**: 显示网名与 fanout
- **-derate**: 显示各延迟的 derate 因子
- **-path_full_clock**: 对 propagated clock 显示时钟网上的单元
- **-path_short**: 从报告中去掉数据通路上的单元行
- **-path_end**: 仅报告端点、数据到达、要求时间与 slack
- **-crosstalk -nets -transition**: SI 延迟报告 (含 delta delay)
- **-pba_mode path | exhaustive**: 路径级重算 (PBA)

A.1.4 Tcl 示例

- **#** 表示注释
- 命令名与开关可缩写 (不必键入全名)
- 反斜杠 \ 用于续行
- 方括号 [] 用于嵌套命令
- 用 **get_clocks** 引用时钟对象

```
report_timing -fall_from [get_clocks PCI_CLK] \

-delay min -max_paths 4

# 用 set_app_var 修改变量

set_app_var sh_enable_page_mode true
```

A.2 PT JOB AID: Run Scripts

PT JOB AID: Run Scripts (PDF p.116)

A.2.1 实用 Tcl 过程 (SolvNet)

过程	说明	Doc ID
aa	always ask: 同时搜索命令与变量	012959
view	在弹出窗口中查看长报告	014947

A.2.2 .synopsys_pt.setup 示例

```
# .synopsys_pt.setup

alias page_on {set_app_var sh_enable_page_mode true}

alias page_off {set_app_var sh_enable_page_mode false}

alias un_on {set_app_var timing_report_unconstrained_paths true}

alias un_off {set_app_var timing_report_unconstrained_paths false}

suppress_message {ENV-003 PTSR-004}

suppress_message CMD-029

history keep 200

set_app_var sh_enable_line_editing true

foreach _file [glob -nocomplain ./tcl_procs/*.tcl] {source $_file}
```

A.2.3 variables.tcl 示例

```
# variables.tcl

set_app_var report_default_significant_digits 4

set_app_var link_create_black_boxes false

set_app_var sh_source_uses_search_path true

set_app_var sh_script_stop_severity E

set_app_var timing_update_status_level high
```

A.2.4 查找非默认应用变量

```
report_app_var -only_changed_vars
```

A.2.5 Path-Based Analysis

```
report_timing -pba_mode path ...

report_timing -pba_mode exhaustive ...
```

A.2.6 RUN.tcl 示例

```
# RUN.tcl

start_profile

set_app_var search_path { . ./scripts ./libs ./designs }

lappend link_path mytech_lib.db RAM_lib.db

source -echo -verbose ./scripts/variables.tcl

read_verilog {top.v A.v B.v}

link_design TOP

read_parasitics ORCA.SPEF.gz

report_annotated_parasitics
```

```
source -echo -verbose constraints.tcl

redirect -tee ./EW.log {check_timing}

report_analysis_coverage

save_session top_savesession

redirect -tee -append ./EW.log {print_message_info}

redirect -tee -append ./EW.log {quit}
```

A.2.7 从 Unix Shell 运行

```
unix% pt_shell -f RUN.tcl | tee -i run.log

# 若脚本异常终止，先修复错误

# 若脚本未正常结束，在 run.log 中搜索 Warning

# 可对消息编号生成 man 页以获取进一步信息
```

A.3 其他 Job Aid 页（PDF p.117–124）

注意

说明：以下各页在原书中为图示速查卡；OCR 无法可靠还原全部版式。
 条目根据 Lab 手册引用、可辨认扫描内容及 PrimeTime Workshop 惯例整理，供实验查阅。
 对照原版 PDF 时请以扫描页为准。

A.3.1 Clocks and More（约 PDF p.117）

Job aid labeled “Clocks and More”（Lab 4 引用）

report_clock

列出全部时钟（含 generated clock）、周期、波形、属性（propagated/ideal）及源 pin/port。

report_clock -attributes

显示时钟附加属性。

report_clock -skew [get_clocks <clk>]

报告时钟不确定性及源延迟（setup/hold、early/late）。

report_clock_timing

时钟网络时序（偏斜、延迟等）；SI 分析可加 -crosstalk。

get_clocks / all_clocks

获取时钟对象集合。

check_timing -override_defaults clock_crossing

报告时钟域交叉（与 GUI Clock Analyzer 矩阵对应）。

set_clock_groups

定义时钟间关系（-asynchronous、-logically_exclusive、-physically_exclusive）。

create_clock / create_generated_clock

创建主时钟与生成时钟。

set_propagated_clock / set_ideal_network

切换 propagated 与 ideal 时钟网络。

A.3.2 Constraints / Interface（约 PDF p.118–119）

约束与接口相关速查（版式以原书图示为准）

- check_timing -verbose: 约束完整性
- report_port -input_delay / report_port -output_delay: I/O 延迟约束
- set_input_delay、set_output_delay: 端口延迟
- set_false_path、set_multicycle_path: 例外路径
- report_constraint -all_violators: 约束违例汇总

A.3.3 PrimeTime SI: Crosstalk Delay Reports（约 PDF p.119–120）

SI 延迟分析速查（Lab 8 引用）

启用 SI 延迟

```
set si_enable_analysis true

read_parasitics -keep_coupling -format SBPF <file>

update_timing
```

report_timing -crosstalk

含 delta delay 的时序报告；常配合 -nets -transition -nosplit。

report_si_bottleneck

串扰瓶颈网；-cost_type 取值示例：

- delta_delay: 最大 delta delay（victim）
- delta_delay_ratio: delta/stage 比值（victim）
- delay_bump_per_aggressor: 相对 VDD 的 aggressor bump 之和

常用开关: `-slack_lesser_than`、`-include_clock_nets`、`-max_nets`。

report_clock_timing -type skew -crosstalk -verbose

最坏时钟偏斜及串扰贡献。

修复思路

`set_coupling_separation` (减小耦合); 加大 victim 驱动或插 buffer; 缩小 aggressor 或隔离网线。

A.3.4 PrimeTime SI Crosstalk Delay: Fine Tuning (PDF p.121)

PrimeTime SI Crosstalk Delay: Fine Tuning for Timing Closure

	主题	Doc ID
SolvNet 文章	在网上生成延时计算报告的简便方法	013383
	有效 aggressor 如何因 bump 大小被筛除	014179
	PrimeTime path-based analysis 精确 sign-off	012134
	在 shell 中用弹出窗口查看长报告 (view)	014947
	用 Tcl 过程 aa 搜索 Synopsys 命令与变量	012959

控制特定网的耦合信息

set_si_delay_analysis

What-If / ECO 相关命令

```
size_cell

report_lib tech_lib buf*

insert_buffer

remove_buffer

set_coupling_separation

set_eco_instance_name_prefix

set_eco_net_name_prefix

estimate_eco

fix_eco_timing

fix_eco_drc

write_changes
```

PBA 相关命令

```

set viol_path [get_timing_paths -slack_lesser_than 0]

set path [get_timing_paths -pba_mode exhaustive $viol_path]

report_timing -crosstalk $path

report_timing -pba_mode path ...

report_timing -pba_mode exhaustive ...

```

调试变量

```

# Variables to control victim reselection

printvar si_xtalk_reselect*

# Variables to control exit criteria

printvar si_xtalk_exit*

# Variable to control electrical filtering

printvar si_filter*

```

A.3.5 PrimeTime SI: Noise Analysis (约 PDF p.122–123)

SI 噪声分析速查 (Lab 9 引用)

流程 si_enable_analysis true → check_noise → update_noise → report_noise

set_noise_parameters

默认开启 between-the-rails 分析与 timing window 滤波 aggressor。

report_noise

-all_violators、-slack_type area | area_percent、-data_pins、-clock_pins、-nworst。

report_noise_calculation

单网详细噪声计算 (-above_low / -below_high、-from、-to)。

set_si_noise_analysis

排除 aggressor 或定义无限 timing window。

set_noise_margin

用户自定义噪声裕量。

report_noise_bumps

按 bump 高度筛选 (如 -bump_greater_than 0.5)。

A.3.6 空白页（PDF p.124）

原书最后一页为刻意留白（*This page was intentionally left blank.*）。